# Morphe

Plataforma de Processamento Digital de Sinais em FPGA
Manual do sistema: funcionamento e estrutura

Antonio Nicassio Santos Lima
Estágio no LabPS/UEFS — supervisão: Prof. Armando S. Sanca

15 de outubro de 2026

**Resumo**

O Morphe é uma plataforma de ensino de Processamento Digital de Sinais. O aluno prepara um sinal num aplicativo no computador, escolhe uma operação, e a operação é executada em ponto fixo na FPGA de uma placa DE1-SoC; o aplicativo mostra lado a lado o resultado do hardware e a referência calculada em ponto flutuante, e a diferença entre os dois é o objeto de estudo.

São cinco as operações em hardware — convolução, filtros FIR e IIR, FFT e IFFT, sobre blocos de 1024 amostras — e sinais de qualquer tamanho são processados por blocos, com técnicas que dão o mesmo resultado de uma operação única. O conversor analógico-digital da placa captura sinais reais, à taxa de amostragem que o aluno escolher, até 200 kHz. A FFT chega a cerca de 92 dB de relação sinal-ruído contra a referência; o IIR coincide bit a bit com o seu modelo; e duas placas atenderam seis clientes simultâneos sem perder nem trocar uma resposta.

Este manual descreve como o sistema funciona e como ele está organizado, para o professor e o monitor que o operam e para quem vai estendê-lo. A plataforma nasceu como Trabalho de Conclusão de Curso de Carlos Valadão na UEFS e foi continuada num estágio curricular no LabPS/UEFS em 2026, que a tornou reproduzível a partir do repositório, utilizável por uma turma e documentada; o capítulo 1 diz o que veio de cada um.

# Sumário


**1 Introdução** **6**
- 1.1 O que é o Morphe . . . 6
- 1.2 Para que serve no ensino de PDS . . . 6
- 1.3 Histórico . . . 7
- 1.4 Visão geral . . . 7
- 1.5 Como ler este manual . . . 8

**2 Arquitetura em três camadas** **9**
- 2.1 As camadas e o que cabe a cada uma . . . 9
- 2.2 O caminho de uma operação . . . 10
- 2.3 Onde está cada camada no repositório . . . 12

**3 O hardware** **13**
- 3.1 A placa DE1-SoC . . . 13
- 3.2 Pontes e mapa de memória . . . 14
- 3.3 Os aceleradores . . . 15
  - 3.3.1 O protocolo comum . . . 16
  - 3.3.2 Convolução e FIR: o `conv1d` . . . 16
  - 3.3.3 FFT e IFFT: o núcleo da Intel . . . 17
  - 3.3.4 IIR: a cascata de seções de segunda ordem . . . 17
- 3.4 A aquisição: o ADC da placa . . . 19
  - 3.4.1 O conversor e o conector . . . 19
  - 3.4.2 Por que um controlador próprio . . . 20
  - 3.4.3 Captura única e captura contínua . . . 20
  - 3.4.4 O que foi validado . . . 21
- 3.5 O codec de áudio . . . 21
  - 3.5.1 Configuração pelo HPS . . . 21
  - 3.5.2 O áudio na FPGA . . . 22
  - 3.5.3 O áudio entre o codec e o computador . . . 22
  - 3.5.4 O que foi validado . . . 23

**4 O servidor na placa** **24**
- 4.1 Estrutura . . . 24
- 4.2 Esperar a FPGA . . . 25
- 4.3 A marca de FPGA preparada . . . 25
- 4.4 Outros cuidados . . . 26
- 4.5 Compilação e início automático . . . 26
- 4.6 Códigos de status . . . 27

**5 O protocolo Morphe-TCP** **28**
5.1 Uma conexão por operação . . . 28
5.2 Os cabeçalhos . . . 28
5.3 As operações . . . 29
5.4 A descoberta: `PING` . . . 31
5.5 O estado do IIR entre execuções . . . 31
5.6 A captura contínua do ADC . . . 31
5.7 Como acrescentar uma operação . . . 32

**6 Representação numérica** **33**
6.1 Os dois formatos . . . 33
6.2 A regra dos 6 dB por bit . . . 34
6.3 A normalização da entrada . . . 34
6.4 O que o núcleo da FFT faz por dentro . . . 35
6.5 Convolução, FIR e IIR . . . 35
6.6 O ADC . . . 36
6.7 Números para referência rápida . . . 36
6.8 Regras práticas . . . 36

**7 O aplicativo cliente** **38**
7.1 Abrir o aplicativo . . . 38
7.2 As janelas . . . 38
7.3 A janela de aquisição . . . 40
7.4 O analisador espectral . . . 41
7.5 Os módulos . . . 42
7.6 O pacote `.mrph` e o comparador . . . 43
7.7 Sinais maiores que o hardware: processamento em blocos . . . 43
7.7.1 Convolução e FIR: *overlap-add* . . . 43
7.7.2 FFT e IFFT: o algoritmo de quatro passos . . . 43
7.7.3 IIR: blocos encadeados pelo estado . . . 44
7.7.4 IIR: blocos com aquecimento (bitstream sem estado) . . . 45
7.7.5 O custo . . . 45

**8 Várias placas, vários alunos** **46**
8.1 Descoberta . . . 46
8.2 A porta de estado . . . 47
8.3 Escolha da placa . . . 47
8.4 A fila e o tempo de resposta . . . 48
8.5 O que foi medido . . . 48
8.6 O que não existe . . . 49
8.7 Sintoma, causa e o que fazer . . . 49

**9 Preparação e operação do laboratório** **51**
9.1 A rotina . . . 51
9.2 O que o `morphe-up.sh` faz . . . 51
9.3 O *tether* da licença . . . 53
9.4 A instalação de turma . . . 54
9.5 As chaves SSH . . . 54
9.6 O servidor no boot da placa . . . 54
9.7 O Quartus em todos os computadores . . . 55
9.8 O aplicativo nos outros computadores . . . 55
9.9 A placa na rede certa . . . 56

9.10 Uma placa nova . . . 57
9.11 Sintoma, causa e o que fazer . . . 57

**10 Desenvolvimento e verificação** **59**
10.1 Três regras antes de começar . . . 59
10.2 Recompilar o bitstream . . . 59
10.3 Testbenches . . . 60
10.4 Testes contra a placa . . . 61
10.5 Sem placa: o servidor em simulação . . . 62
10.6 A proveniência . . . 62
10.7 Fluxo de trabalho com o `git` . . . 62
10.8 Acrescentar um acelerador . . . 62

**11 Limitações e trabalho futuro** **64**
11.1 Hardware . . . 64
11.2 Aquisição . . . 64
11.3 Codec de áudio . . . 65
11.4 Infraestrutura . . . 66
11.5 Ideias que a plataforma já permite . . . 66

**A Estrutura do repositório** **67**

**B Referência rápida** **69**

# Lista de Figuras


1.1 Visão geral: os computadores do laboratório falam com as placas pela rede (TCP, porta 5000); a estação programa a FPGA pelo USB-Blaster e sobe o servidor por SSH. . . . . 8

2.1 As três camadas e os módulos de cada uma. . . . . 9
2.2 O caminho de uma operação, do clique na janela ao gráfico. . . . . 11

3.1 Mapa de memória visto pelo servidor: memórias de dados atrás da ponte HPS→FPGA e registradores de controle atrás da ponte leve. Os endereços são deslocamentos a partir da base de cada ponte. . . . . 14
3.2 O que há dentro da FPGA: quatro aceleradores e o controlador do ADC, cada um com as suas memórias e os seus registradores de controle. . . . . 15
3.3 O bloco IIR: cascata de seções de segunda ordem (acima) e a seção em forma direta I (abaixo). Um só multiplicador-acumulador atende todas as seções, dois ciclos por seção. . . . . 18
3.4 O caminho da aquisição: o controlador próprio dispara cada conversão do LTC2308 a $1/f_s$ e grava a amostra na memória de captura, que o servidor lê pela ponte. . . . . 19

5.1 Os cabeçalhos de requisição e de resposta, em bytes. . . . . 28

6.1 Os dois formatos de ponto fixo, bit a bit. . . . . 33

7.1 *Overlap-add*: cada bloco de $L$ amostras é convoluído na placa; as caudas de $M-1$ amostras se somam. . . . . 44
7.2 FFT de $N = M \cdot 1024$ pontos pelo algoritmo de quatro passos: $M$ FFTs de 1024 na placa, fatores de giro e DFTs de $M$ pontos no PC. . . . . 44
7.3 IIR por blocos: cada bloco começa algumas amostras antes (*aquecimento*) e essa parte da saída é descartada. . . . . 45

8.1 O método antigo de escolha, que vale quando nenhuma placa responde na porta de estado: o cliente sonda todas em paralelo e fica com a que responder primeiro. Uma placa ocupada só responde ao fim da operação em curso. . . . . 48

9.1 O que o `morphe-up.sh` faz, em ordem. . . . . 52

# Lista de Tabelas


1.1 O que o estágio acrescentou ao projeto original. . . . . 7

2.1 Responsabilidades de cada camada. . . . . 9

3.1 Recursos da FPGA ocupados pelo projeto (compilação de 24/09/2026). . . . . 13
3.2 Os registradores do controlador do ADC, na ponte leve, e a sua memória. . . . . 21
3.3 Os dois conversores da placa. . . . . 21

4.1 Códigos de status das respostas . . . . . 27

5.1 As operações do protocolo e o que vai em cada *payload*. Todos os valores ocupam 4 bytes. . . . . 30

6.1 Os formatos de ponto fixo e onde cada um é usado. . . . . 33
6.2 FFT de um sinal de pico 3,3 com escalas crescentes, comparada com `np.fft.fft` (placa, 10/09/2026). . . . . 34
6.3 Efeito da normalização, medido na placa (10/09/2026). . . . . 34
6.4 Erros medidos na placa, para comparação. . . . . 36

7.1 Os módulos do cliente, em `Python/`. . . . . 42

8.1 Vários clientes simultâneos, operações de 1024 amostras, metade convoluções (≈215 ms) e metade FFTs (≈21 ms). Placa livre: mediana de 120 ms (22/09/2026). 48
8.2 Sintomas ligados a várias placas e vários clientes. . . . . 50

9.1 As opções do `morphe-up.sh`. . . . . 53
9.2 Sintomas da operação do laboratório. . . . . 58

10.1 Os testes contra a placa. . . . . 61

A.1 A documentação técnica, em `docs/` e na raiz. . . . . 68

# Capítulo 1

# Introdução

## 1.1 O que é o Morphe

O Morphe é uma plataforma para executar operações de Processamento Digital de Sinais (PDS) em hardware e comparar o resultado com o que o computador calcula. O aluno prepara um sinal numa janela do aplicativo, escolhe a operação e recebe de volta duas respostas lado a lado: a que a FPGA de uma placa DE1-SoC calculou em ponto fixo e a que o próprio computador calculou em ponto flutuante. A diferença entre as duas é o objeto de estudo.

São cinco as operações executadas em hardware:

**Convolução** linear de dois sinais de até 1024 amostras cada, com saída de até 2047 amostras.

**Filtro FIR** a mesma convolução, com a resposta ao impulso vinda de um projetista de filtros por especificação.

**Filtro IIR** cascata de até 16 seções de segunda ordem (ordem até 32), também com projetista próprio.

**FFT** transformada de Fourier discreta de 1024 pontos.

**IFFT** a transformada inversa, no mesmo núcleo da FFT.

Além delas, a plataforma **adquire sinais**: o conversor analógico-digital da placa captura um sinal real, de um gerador de funções ou de um sensor, à taxa de amostragem que o aluno escolher, de 1 a 200 kHz, constante durante toda a captura, e a captura pode ser usada como entrada de qualquer uma das cinco operações (seção 3.4).

O hardware trabalha com blocos de tamanho fixo (1024 amostras), mas o aplicativo aceita sinais de qualquer tamanho: divide o sinal em blocos, manda cada um à placa e recompõe o resultado, com técnicas que dão a mesma resposta que uma operação única daria (capítulo 7).

## 1.2 Para que serve no ensino de PDS

Nas disciplinas de PDS as operações são estudadas com números reais, e o computador as calcula em ponto flutuante de 64 bits, que para quase todo fim didático é o mesmo que calcular com números reais. Um circuito digital dedicado não trabalha assim: usa ponto fixo, com um número pequeno de bits para a parte fracionária, e cada multiplicação precisa ser arredondada ou truncada.

O Morphe torna essa diferença visível e mensurável. Com ele o aluno pode:

- ver o erro de quantização de uma FFT em decibéis e observar que ele melhora 6 dB a cada bit de faixa aproveitada (capítulo 6);

- provocar uma saturação ou um estouro de acumulador e reconhecer o sintoma na saída;
- comparar um filtro IIR projetado em ponto flutuante com o mesmo filtro executado em ponto fixo, e entender por que o hardware usa seções de segunda ordem e não um polinômio de ordem alta;
- medir quanto tempo cada operação leva na placa e relacionar esse tempo com o número de operações do algoritmo;
- capturar um sinal de verdade pelo conversor da placa, ver o *aliasing* de um tom acima de $f_s/2$ e medir quantos bits efetivos o conversor entrega.

## 1.3 Histórico

O Morphe nasceu como Trabalho de Conclusão de Curso de Carlos Valadão na UEFS, *Implementação de Ferramentas de Computação Numérica para o Processamento Digital de Sinais em Hardware Reconfigurável*. A arquitetura, o núcleo de convolução `conv1d`, a integração do núcleo de FFT, o servidor em C e o cliente gráfico em Python são dele, e o projeto é distribuído sob a licença GNU GPL versão 3.

A partir de setembro de 2026 a plataforma foi continuada num estágio curricular no LabPS/UEFS, por Antonio Nicassio Santos Lima, sob supervisão do Prof. Armando S. Sanca. O objetivo do estágio foi tornar a plataforma reproduzível a partir do repositório e utilizável por uma turma de alunos que não conhece os detalhes de baixo nível. A Tabela 1.1 resume o que mudou; o registro dia a dia está em `docs/DIARIO.md` e as correções feitas no projeto original, com os motivos, em `docs/RESSALVAS.md`.

Tabela 1.1: O que o estágio acrescentou ao projeto original.

| | **Projeto original** | **Hoje** |
|---|---|---|
| Tamanho | 128 amostras na convolução e no FIR | 1024 amostras em todas as operações, e sinais maiores por blocos |
| Operações | convolução, FIR, FFT | as três, mais IFFT e IIR |
| Sinais | gerados na janela | gerados, lidos de arquivo, ou capturados pelo ADC da placa, à taxa escolhida |
| Filtros | sem projetista por especificação | projetistas FIR e IIR por especificação |
| Precisão | entrada da FFT sem normalização | normalização da entrada: de 50 para 92 dB de SNR na FFT |
| Preparar a placa | passos manuais, conhecidos pelo autor | um comando, `morphe-up.sh` |
| Laboratório | uma estação, uma placa | duas placas, vários clientes ao mesmo tempo, com o estado de cada placa à vista; o aplicativo e o Quartus em todos os computadores |
| Reprodutibilidade | o que funcionava estava só na estação | tudo sai do repositório, com manifesto de integridade |

## 1.4 Visão geral

A Figura 1.1 é a figura de referência deste manual: todo o resto detalha uma das partes dela.

Cada **PC do laboratório** roda o aplicativo Morphe, escrito em Python. Ele não precisa de nenhum software da Intel: fala com as placas só pela rede.

Cada **placa DE1-SoC** tem, no mesmo chip, um processador ARM rodando Linux (o HPS, *Hard Processor System*) e uma FPGA. No processador roda o `morphe_server`, que recebe as

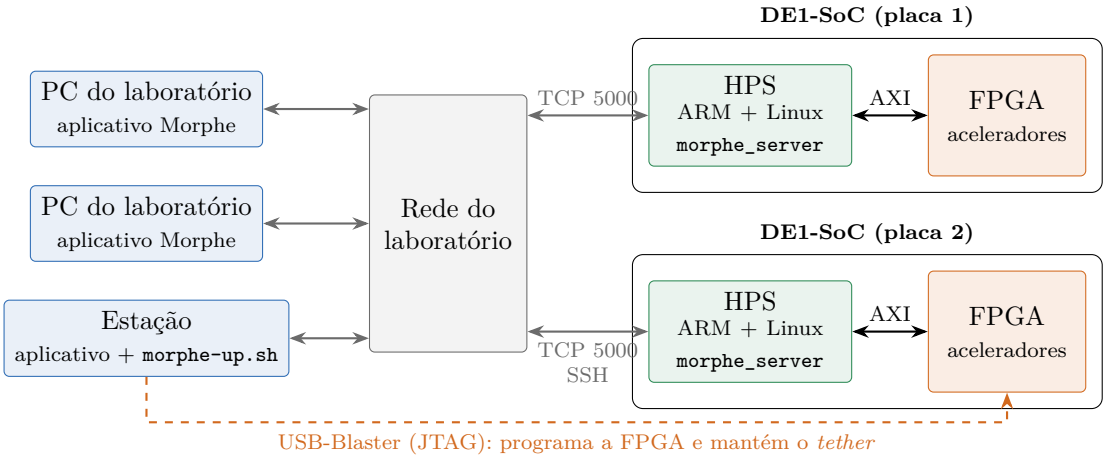



Figura 1.1: Visão geral: os computadores do laboratório falam com as placas pela rede (TCP, porta 5000); a estação programa a FPGA pelo USB-Blaster e sobe o servidor por SSH.

requisições pela rede na porta 5000; na FPGA estão os aceleradores que fazem as contas. Os dois lados conversam por pontes internas ao chip (seção 3.2).

A **estação** é o computador ligado às placas pelo cabo USB-Blaster. É dela que se roda o `morphe-up.sh`, que programa a FPGA pelo JTAG, compila e reinicia o servidor na placa por SSH e registra a placa para os clientes. A estação também roda o aplicativo, como qualquer outro PC.

## 1.5 Como ler este manual

Quem só vai **usar** a plataforma precisa do capítulo 7 (o aplicativo) e do capítulo 9 (preparar as placas). Quem vai **estender** a plataforma — acrescentar uma operação, mudar um acelerador, recompilar o hardware — deve ler os capítulos em ordem: a arquitetura (2), o hardware (3), o servidor (4), o protocolo (5) e a representação numérica (6) são o que se precisa saber antes de mexer em qualquer um deles.

Convenções usadas no texto:

- nomes de arquivos, comandos e sinais do hardware aparecem em `fonte monoespaçada`; caminhos são relativos à raiz do repositório;
- endereços e constantes do hardware aparecem em hexadecimal, com o prefixo `0x`;
- Q$m.n$ é um número em ponto fixo com sinal, $m$ bits de parte inteira e $n$ de parte fracionária (capítulo 6);
- todo número medido vem com a condição em que foi medido. Um número sem condição é de especificação, não de medida.

# Capítulo 2

# Arquitetura em três camadas

## 2.1 As camadas e o que cabe a cada uma

O Morphe se divide em três camadas, uma por máquina ou por tipo de circuito (Figura 2.1). Cada uma é escrita numa linguagem e tem uma responsabilidade que as outras não repetem.

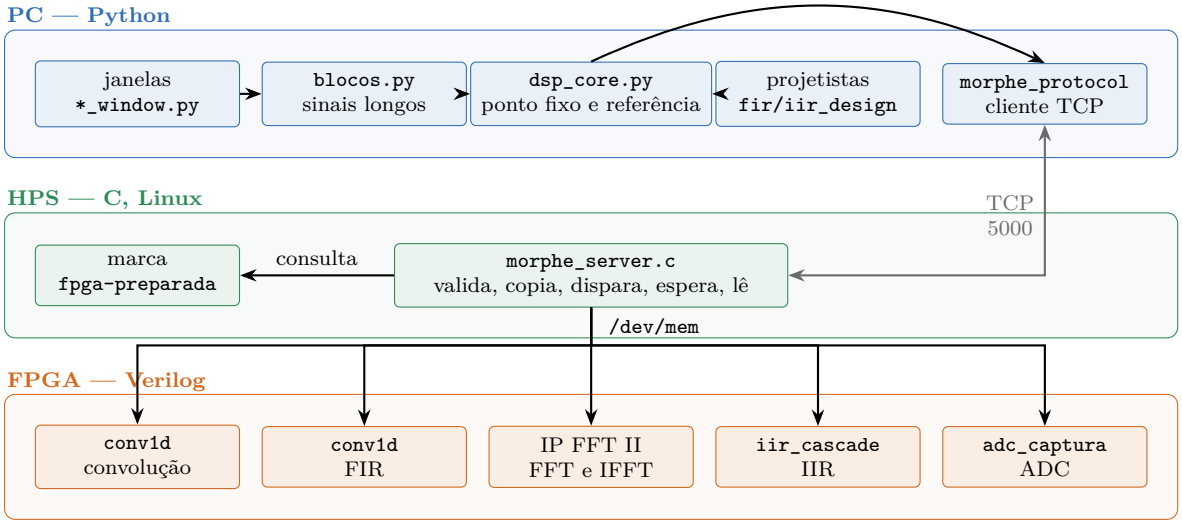



Figura 2.1: As três camadas e os módulos de cada uma.

Tabela 2.1: Responsabilidades de cada camada.

| | **Faz** | **Não faz** |
|---|---|---|
| **Cliente**<br>Python, no PC | gera ou lê o sinal; projeta filtros; converte para ponto fixo; divide sinais longos em blocos; calcula a referência em ponto flutuante; compara e mostra | não sabe endereços de memória nem detalhes da FPGA |
| **Servidor**<br>C, no HPS da placa | valida a requisição; copia os dados para as memórias da FPGA; dispara o acelerador; espera; devolve o resultado | não converte formatos numéricos (com uma exceção na FFT, seção 3.3.3); não guarda nada entre requisições |
| **Aceleradores**<br>Verilog, na FPGA | fazem a conta, em ponto fixo, sobre blocos de tamanho fixo | não conhecem a rede nem o tamanho do sinal que o aluno pediu |

### Três princípios que atravessam as camadas

**O servidor não guarda estado.** Cada operação chega completa — os dados e todos os parâmetros — e sai completa. Não há sessão, reserva de placa nem memória da requisição anterior. É isso que permite que vários clientes usem a mesma placa intercalando operações, e que um cliente possa mudar de placa de uma operação para a outra sem nada a transferir.

**O ponto fixo é decidido no cliente.** O cliente converte o sinal para o formato do acelerador (Q15.16 ou Q15.8) antes de enviar, e converte o resultado de volta. O servidor copia inteiros de 32 bits sem interpretá-los, com uma única exceção: aplicar o expoente de bloco da FFT, que só ele conhece (seção 3.3.3). Fora isso, toda decisão numérica — normalizar a entrada, onde saturar, que fator aplicar à IFFT — mora num lugar só, em Python, onde é fácil de ler e de testar.

**O hardware tem tamanho fixo; o software se adapta.** Os aceleradores foram sintetizados para 1024 amostras. O servidor completa com zeros o que o cliente não mandou e devolve só as amostras que fazem sentido; o cliente, por sua vez, divide sinais maiores em blocos de 1024. Nenhum dos dois lados exige que o aluno pense em 1024.

### Os números de configuração existem em dois lugares

Os limites do hardware — 1024 amostras, 16 seções de IIR, os bits fracionários de cada formato — estão escritos em `C/morphe_config.h`, para o servidor, e repetidos em `Python/morphe_config.py`, para o cliente. Os dois arquivos precisam mudar juntos. Os endereços das memórias e registradores, ao contrário, não são escritos à mão: vêm do `C/hps_0.h`, gerado a partir do projeto do hardware (seção 3.2).

## 2.2 O caminho de uma operação

A Figura 2.2 acompanha uma convolução do clique na janela até o gráfico. As outras operações seguem o mesmo roteiro, mudando só as memórias e o formato numérico.

1. **A janela entrega os sinais.** $x[n]$ e $h[n]$ chegam ao módulo de protocolo como vetores em ponto flutuante, do tamanho que o aluno escolheu.

2. **O cliente codifica.** Cada amostra vira um inteiro de 32 bits em Q15.16: $q = \mathrm{round}(v \cdot 2^{16})$, saturado na faixa do formato. Se o sinal passa de 1024 amostras, a divisão em blocos acontece antes deste passo, e os passos seguintes se repetem por bloco.

3. **Uma conexão por operação.** O cliente abre uma conexão TCP com a placa, envia um cabeçalho de 20 bytes (operação, tipo dos dados, tamanhos) e os dados logo em seguida (capítulo 5).

4. **O servidor valida.** Confere a assinatura e a versão do protocolo, a operação, os tamanhos contra os limites do hardware e se a FPGA foi preparada pelo `morphe-up.sh` (capítulo 4). Qualquer falha vira uma resposta de erro com uma mensagem legível, e a FPGA não é tocada.

5. **O servidor escreve nas memórias.** Copia $x$ e $h$ para as memórias de entrada do acelerador, através da ponte HPS→FPGA, completando com zeros até 1024 amostras.

6. **O servidor dispara.** Escreve 1 no registrador `start` do acelerador, através da ponte leve. O acelerador detecta a subida desse sinal e começa.

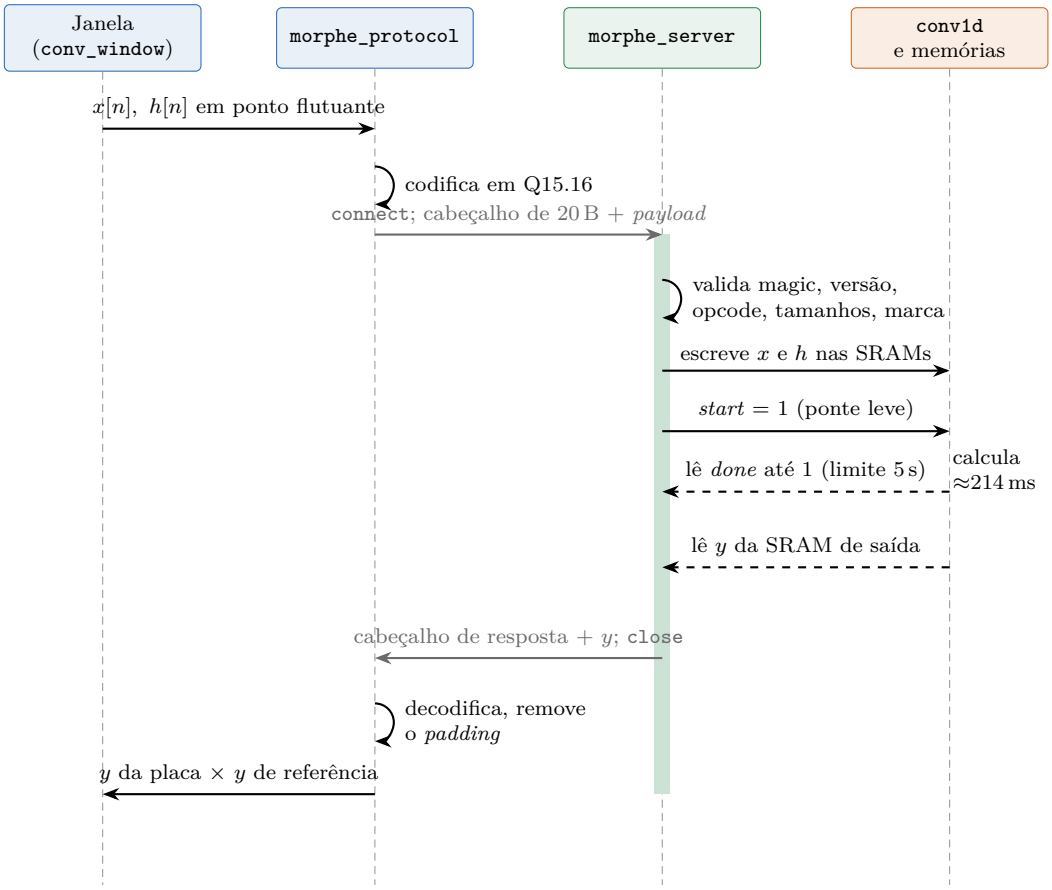



Figura 2.2: O caminho de uma operação, do clique na janela ao gráfico.

7. **O acelerador calcula** e, ao terminar, leva o sinal `done` a 1. A convolução de 1024 por 1024 amostras leva cerca de 214 ms; a FFT de 1024 pontos, cerca de 21 ms (tempos de ida e volta medidos pelo `morphe_ping` em 15/09/2026, com a placa sem outros clientes).

8. **O servidor espera.** Lê o `done` a cada 50 µs. Se ele não subir em 5 s, a operação é abandonada com o erro de tempo esgotado — o sintoma típico de um bitstream que parou de funcionar (seção 3.3).

9. **O servidor lê e responde.** Copia as $n_x + n_h - 1$ amostras úteis da memória de saída, devolve o `start` a 0 e envia o cabeçalho de resposta e os dados. Fecha a conexão.

10. **O cliente decodifica e compara.** Converte de volta para ponto flutuante, descarta o que for enchimento de zeros e calcula a referência em ponto flutuante com o NumPy. A janela mostra as duas e o erro entre elas.

Dois fatos decorrem desse roteiro e aparecem na prática.

O tempo de uma operação **não depende do tamanho do sinal pedido**: o acelerador sempre processa os blocos completos de 1024 amostras, e os zeros de enchimento custam o mesmo que amostras de verdade. Pela construção do `conv1d`, uma convolução de 10 por 10 amostras custa o mesmo que uma de 1024 por 1024.

Como o servidor atende **uma conexão de cada vez**, duas pessoas na mesma placa não recebem erro: a segunda espera a primeira terminar, e o tempo de cada uma soma. O capítulo 8 trata disso.

## 2.3 Onde está cada camada no repositório

| **Camada** | **Onde** |
|---|---|
| Cliente | `Python/` (aplicativo e módulos); testes em `Python/ferramentas/` |
| Servidor | `C/` (`morphe_server.c` e cabeçalhos) |
| Aceleradores | `Quartus/` (RTL, projeto do Platform Designer, bitstream) |
| Operação | `morphe-up.sh`, `instala-quartus.sh`, `deploy.sh`, na raiz |

A árvore completa está no apêndice A.

# Capítulo 3

# O hardware

## 3.1 A placa DE1-SoC

A DE1-SoC, da Terasic, é construída em torno de um Cyclone V SoC da Intel (`5CSEMA5F31C6`). O "SoC" do nome quer dizer que o chip reúne duas coisas bem diferentes:

**HPS** (*Hard Processor System*): um processador ARM Cortex-A9 de dois núcleos, com memória e periféricos próprios, que roda um Linux gravado no cartão SD da placa. No Morphe ele cuida da rede e roda o servidor.

**FPGA** lógica programável, configurada pelo bitstream que o Quartus gera. No Morphe ela contém os aceleradores e as memórias em que eles trabalham.

Os dois lados estão no mesmo chip e se comunicam por pontes internas, sem passar por nenhum fio da placa. É essa proximidade que permite ao servidor escrever mil amostras numa memória da FPGA como se escrevesse num vetor em C.

A lógica da FPGA roda com o relógio de 50 MHz da placa (`CLOCK_50`). A análise de tempo da última compilação (24/09/2026, com o ADC) mede uma frequência máxima de 61,56 MHz para esse domínio: há folga de cerca de 20 %. A Tabela 3.1 mostra quanto do chip o projeto ocupa.

Tabela 3.1: Recursos da FPGA ocupados pelo projeto (compilação de 24/09/2026).

| **Recurso** | **Usado** | **Disponível** | **Ocupação** |
|---|---:|---:|---:|
| Elementos lógicos (ALMs) | 12 621 | 32 070 | 39 % |
| Blocos DSP (multiplicadores) | 30 | 87 | 34 % |
| Memória interna (bits) | 2 220 342 | 4 065 280 | 55 % |

A memória de captura do ADC, de 32 768 palavras, é o que fez a ocupação de memória subir de 42 % para 55 % em relação à compilação anterior (15/09/2026, sem o ADC). Ainda sobra quase metade.

### O bitstream e a licença da FFT

O bitstream versionado é `Quartus/output_files/soc_system_time_limited.sof`, compilado com o Quartus Prime Lite 20.1. O núcleo de FFT é um IP da Intel que a edição Lite só permite usar em modo de avaliação: o bitstream funciona **enquanto o programador JTAG continuar conectado** à placa — o *tether* — e para depois de cerca de uma hora sem ele. Isso vale para o bitstream inteiro, não só para a FFT: sem o *tether*, a convolução também passa a esgotar o tempo. O `morphe-up.sh` mantém o *tether* vivo (capítulo 9).

## 3.2 Pontes e mapa de memória

O HPS enxerga a FPGA como faixas de endereços. O Morphe usa duas pontes (Figura 3.1):

**Ponte HPS→FPGA** (`h2f_axi_master`), a partir de `0xC000 0000`. Larga e rápida, é por onde passam os dados: as memórias de entrada e de saída de cada acelerador.

**Ponte leve** (*lightweight*), a partir de `0xFF20 0000`. Estreita, feita para registradores de controle: os PIOs de `start`, `done` e de parâmetros.

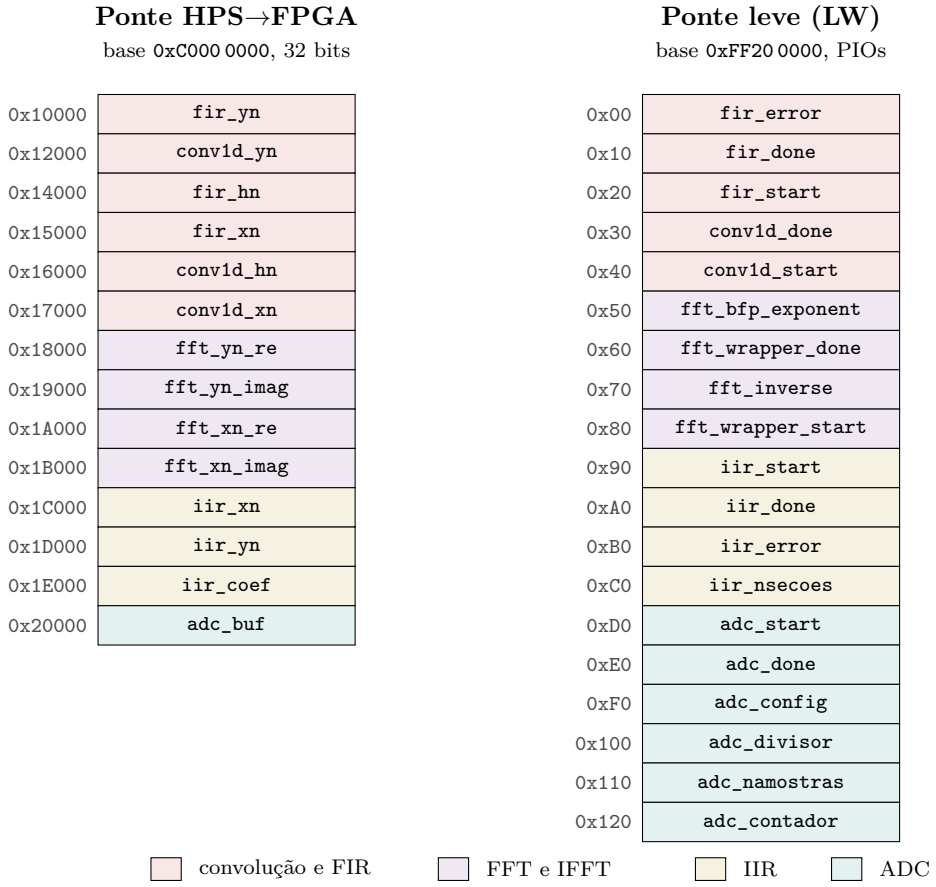



Figura 3.1: Mapa de memória visto pelo servidor: memórias de dados atrás da ponte HPS→FPGA e registradores de controle atrás da ponte leve. Os endereços são deslocamentos a partir da base de cada ponte.

**As memórias têm duas portas.** Cada uma das catorze memórias de dados é uma RAM interna da FPGA de porta dupla: uma porta está ligada à ponte, para o HPS, e a outra ao acelerador (ou ao controlador do ADC). O servidor e o acelerador nunca disputam a mesma porta; o que os separa no tempo é o protocolo de `start` e `done` (seção 3.3). As memórias de entrada guardam 1024 palavras de 32 bits (4 KB); as de saída da convolução e do FIR, 2047 (8 KB); a de coeficientes do IIR, 5 palavras por seção para até 16 seções; a de captura do ADC, 32 768 palavras (128 KB).

**Como o servidor chega lá.** O servidor abre `/dev/mem` e mapeia as duas faixas no próprio espaço de endereços com `mmap`; a partir daí, escrever `g_conv_xn[i]` é escrever na memória da FPGA. Por isso ele precisa rodar como `root`. O tamanho da faixa mapeada é calculado em

tempo de compilação a partir do `hps_0.h`, e verificações estáticas no código recusam compilar se alguma memória for menor do que os limites de `morphe_config.h` exigem.

**O `hps_0.h` é gerado, nunca editado.** Os endereços da Figura 3.1 são decididos no Platform Designer, a ferramenta do Quartus que monta o sistema, e ficam registrados no arquivo `soc_system.sopcinfo`. O `Quartus/gen_hps_header.py` lê esse arquivo e escreve o `C/hps_0.h`; com `--check`, apenas confere se o cabeçalho versionado ainda corresponde ao projeto. Um servidor compilado com endereços velhos escreve nos lugares errados sem dar erro nenhum; a conferência existe para que isso não aconteça.

**Cuidado com o mapa do "DE1-SoC Computer".** Os exemplos da Intel para a placa usam `0xC800 0000` como base da memória da FPGA. No Morphe a base é `0xC000 0000`. Código copiado desses exemplos lê e escreve fora das memórias do projeto (`docs/RESSALVAS.md`, item 11).

## 3.3 Os aceleradores

A Figura 3.2 mostra o que há dentro da FPGA: quatro aceleradores e o controlador de captura do ADC, cada um com as suas memórias e os seus registradores de controle, todos ligados ao HPS pela interconexão que o Platform Designer gera.

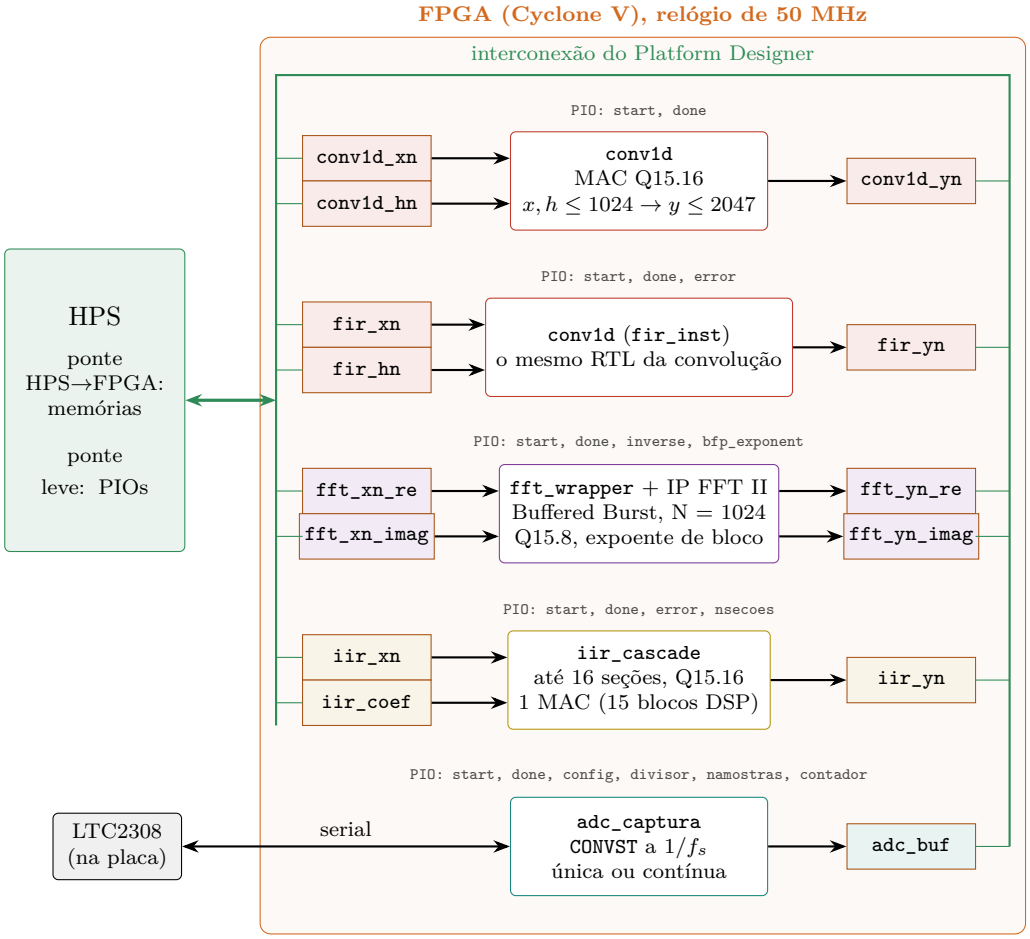



Figura 3.2: O que há dentro da FPGA: quatro aceleradores e o controlador do ADC, cada um com as suas memórias e os seus registradores de controle.

### 3.3.1 O protocolo comum

Todos os aceleradores obedecem ao mesmo combinado com o servidor:

1. com `start` em 0, o servidor escreve as entradas nas memórias;
2. o servidor leva `start` a 1; o acelerador detecta a *subida* do sinal, zera o `done` e começa;
3. o acelerador lê as entradas, calcula, escreve as saídas e leva `done` a 1;
4. o servidor, que lê `done` a cada 50 µs, lê as saídas e devolve `start` a 0.

Como o acelerador reage à subida e não ao nível, um `start` esquecido em 1 não dispara uma segunda execução. E como o servidor zera todos os `start` na primeira requisição depois de a FPGA ser preparada, um valor que tenha sobrado de antes do bitstream do Morphe também não dispara nada.

Os aceleradores acessam as memórias por pequenos controladores que gastam alguns ciclos por acesso. É um custo deliberado de simplicidade: numa convolução, o tempo é dominado por esses acessos, não pela multiplicação. Os controladores estão em `memory_read_controller.v` e em `memory_write_controller.v`.

**O tempo como sinal de saúde.** Uma convolução de 1024 por 1024 amostras executa cerca de 8,4 milhões de ciclos, uns 168 ms a 50 MHz; de ponta a ponta, com a rede, o cliente mede 214 ms. Uma resposta muito mais rápida que isso é sinal de defeito, não de sorte: um bitstream mal compilado já foi visto levantar o `done` em 3,3 ms sem ter calculado nada (`docs/RESSALVAS.md`, item 13).

### 3.3.2 Convolução e FIR: o `conv1d`

O `conv1d.v` calcula a convolução linear

$$y[n] = \sum_k x[k]\, h[n-k], \qquad n = 0, 1, \ldots, 2046,$$

com os limites da soma ajustados a cada $n$ para não ler fora dos vetores. É uma máquina de estados com um único multiplicador-acumulador: para cada $n$, lê $x[k]$ e $h[n-k]$, multiplica, acumula e, ao fim da soma, escreve $y[n]$.

A aritmética é em Q15.16. O produto de dois números de 32 bits tem 64 bits (Q32.32), e a soma corre com os produtos *inteiros*, num acumulador de 64 bits. Só ao escrever $y[n]$ o resultado volta a Q15.16, *arredondado* uma única vez:

$$y[n] = \left\lfloor \frac{\sum_k x[k]\, h[n-k]}{2^{16}} + \frac{1}{2} \right\rfloor,$$

meio para cima também nos negativos, a mesma regra do IIR. O erro contra a conta exata é de no máximo meio LSB ($7{,}6 \times 10^{-6}$), qualquer que seja o número de termos. A saída são os bits 47 a 16 do acumulador e *não satura*: com sinais de amplitude $A$ e $N$ termos, ela estoura quando $NA^2 \geq 32768$ — com $N = 1024$, a partir de $A \approx 5{,}7$ — e aparece com o sinal trocado. Uma saída negativa para sinais positivos é esse estouro, não um defeito.

**Antes era truncamento.** Até 14/10/2026 o `conv1d` cortava cada produto para Q15.16 (bits 47 a 16) *antes* de somar, como no TCC original: um erro de até um LSB por termo, sempre para baixo, que numa convolução de 1024 por 1024 chegava a 521 LSB ($8 \times 10^{-3}$). A troca pelo arredondamento foi feita em 15/10/2026 e medida na placa: 1024 por 1024 amostras aleatórias em $\pm 2$ deram erro máximo de **0,500 LSB** na convolução e no FIR, e 137 dB de SNR contra o `np.convolve` em ponto flutuante de 64 bits. O testbench `Quartus/testbench/roda_tb_conv.sh` confere as 2047 saídas contra a conta exata.

**O FIR é o mesmo circuito.** O filtro FIR é uma segunda instância do `conv1d` (`fir_inst`), com memórias próprias. Filtrar um sinal por um FIR é convoluí-lo com a resposta ao impulso do filtro, e é exatamente isso que acontece; a diferença entre as duas operações está no aplicativo, que oferece para o FIR um projetista de filtros. O projeto original trazia também um IP de FIR da Intel (`fir_ii`) e o seu adaptador, `fir_wrapper.v`, com a instância comentada no `ghrd_top.v`; os dois saíram do projeto em 01/10/2026 e continuam no histórico do git. O registrador `fir_error` sobrou desse adaptador e o `conv1d` não o aciona.

### 3.3.3 FFT e IFFT: o núcleo da Intel

A transformada usa o IP *FFT II* da Intel, configurado para 1024 pontos em modo *Buffered Burst*, com entrada de 24 bits e fatores de giro de 18 bits. O `fft_wrapper.v` faz a ponte entre as memórias e o IP: lê as partes real e imaginária da entrada, entrega-as ao IP amostra por amostra respeitando o *handshake* do barramento de fluxo, recebe o resultado e o escreve nas memórias de saída. O registrador `fft_inverse` escolhe entre a transformada direta e a inversa; o resto do caminho é o mesmo.

Duas particularidades desse núcleo aparecem no resultado:

**Expoente de bloco.** A cada estágio, o IP detecta se os valores cresceram e, se preciso, desloca as 1024 amostras *juntas*, acumulando um expoente único para o bloco (*block floating point*). O adaptador captura esse expoente no registrador `fft_bfp_exponent`, e o servidor o aplica, junto com o fator $2^{-8}$ do formato Q15.8, antes de devolver o resultado em ponto flutuante. É a única conversão numérica feita no servidor.

**IFFT sem o fator $1/N$.** O núcleo entrega a inversa não normalizada, $N$ vezes a IDFT matemática. O cliente aplica o $1/1024$.

O capítulo 6 explica o que o expoente de bloco e os 24 bits de entrada significam para a precisão, e por que ela para em torno de 92 dB.

### 3.3.4 IIR: a cascata de seções de segunda ordem

O filtro IIR é a única operação em que a saída depende das saídas anteriores, e isso muda o que importa no projeto do circuito. Ele é implementado como uma cascata de até 16 seções de segunda ordem (*biquads*), cada uma em forma direta I (Figura 3.3):

$$y_s[n] = b_0\,x_s[n] + b_1\,x_s[n-1] + b_2\,x_s[n-2] - a_1\,y_s[n-1] - a_2\,y_s[n-2],$$

com a saída de cada seção servindo de entrada para a seguinte. Os coeficientes vêm da memória `iir_coef`, cinco por seção, na ordem $b_0, b_1, b_2, a_1, a_2$ ($a_0 = 1$ fica implícito); o número de seções, do registrador `iir_nsecoes`.

Cada escolha desse projeto foi medida antes de ser feita:

**Seções de segunda ordem, e não um polinômio de ordem $N$.** A posição dos polos de um polinômio de ordem alta é extremamente sensível aos coeficientes. Um Butterworth de ordem 22, estável por construção, volta do próprio MATLAB com um polo de módulo 1,278 — instável — porque achar as raízes daquele polinômio já é mal condicionado em ponto flutuante de 64 bits. A mesma especificação em seções de segunda ordem tem o maior polo em 0,988.

**Forma direta I.** Tem um acumulador único e largo e arredonda uma vez só, na saída da seção. A forma direta II transposta, melhor em ponto flutuante, guarda estados já arredondados, e numa realimentação esse erro circula.

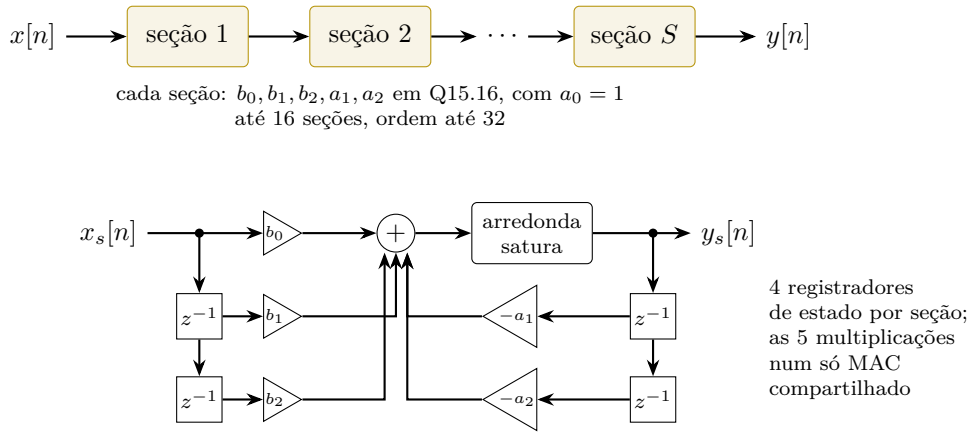



Figura 3.3: O bloco IIR: cascata de seções de segunda ordem (acima) e a seção em forma direta I (abaixo). Um só multiplicador-acumulador atende todas as seções, dois ciclos por seção.

**Arredondar, e não truncar.** Truncar introduz um viés de meio LSB negativo por amostra. Num filtro realimentado esse viés não some: vira um deslocamento DC de $-0{,}5/(1+a_1+a_2)$ LSB na saída. No FIR o viés não realimenta, mas soma um erro que cresce com o número de termos; por isso, desde 15/10/2026, o `conv1d` também arredonda (seção 3.3.2).

**Q15.16, o mesmo formato do `conv1d`.** É suficiente: o raio dos polos se desloca no máximo cerca de $10^{-6}$ ao quantizar, mesmo com polos em $|p| = 0{,}9984$.

**Uma amostra atravessa todas as seções antes da próxima.** Assim bastam quatro registradores de estado por seção e uma única passagem pela memória. O resultado é idêntico, bit a bit, ao de processar seção por seção, porque cada seção só vê a saída já arredondada da anterior.

**Um só multiplicador-acumulador, compartilhado.** As cinco multiplicações de 32 por 32 bits de uma seção são feitas juntas, e o mesmo circuito atende as 16 seções em sequência. O custo em blocos DSP (15, porque cada multiplicação de 32 bits ocupa três multiplicadores de 27 bits) não depende do número de seções. Cada seção leva dois ciclos: um para selecionar os operandos, outro para calcular. Com um ciclo só, o circuito não fechava a 50 MHz (49,11 MHz medidos em 14/09/2026); com dois, fecha a 60,99 MHz.

Se alguma amostra satura, o registrador `iir_error` sobe; o resultado é devolvido mesmo assim, saturado, e o cliente avisa o aluno. O estado interno é zerado a cada execução, para que o mesmo sinal dê sempre a mesma saída.

**Verificado bit a bit.** O RTL do IIR tem um modelo de referência em Python que usa inteiros de precisão arbitrária (o acumulador de uma seção chega a $2^{65}$, além do que um `float64` representa exatamente). O testbench em Icarus Verilog (`Quartus/testbench/roda_tb_iir.sh`) e a placa (`testa_iir_hw.py`) são comparados com esse modelo e precisam coincidir em todas as amostras: num filtro realimentado, divergir um LSB numa amostra é divergir para sempre dali em diante, então "parecido" não serve de critério. Esse modelo faz exatamente a conta do hardware, e por isso o erro contra ele é zero *por construção*: ele prova que o circuito está certo, mas não mede quanto o ponto fixo custa. Para isso o aplicativo compara a saída da placa com a mesma cascata em ponto flutuante de 64 bits (seção 7.2).

## 3.4 A aquisição: o ADC da placa

A placa tem um conversor analógico-digital, e desde 24/09/2026 o Morphe o usa para capturar sinais. O aluno escolhe a taxa de amostragem, de 1 a 200 kHz, e o hardware a mantém constante durante a captura, amostra por amostra. A Figura 3.4 mostra o caminho, do conector até a memória que o servidor lê.

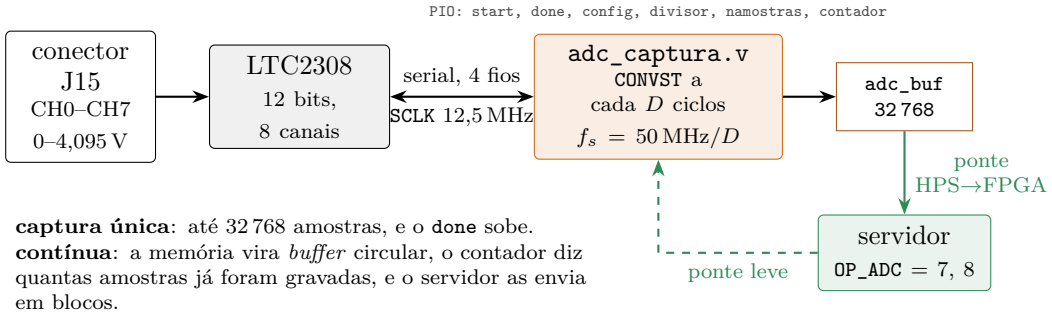



Figura 3.4: O caminho da aquisição: o controlador próprio dispara cada conversão do LTC2308 a $1/f_s$ e grava a amostra na memória de captura, que o servidor lê pela ponte.

### 3.4.1 O conversor e o conector

O conversor é o LTC2308, da Analog Devices (antiga Linear): 12 bits, 8 canais, por aproximações sucessivas, até 500 mil amostras por segundo, ligado à FPGA por uma interface serial de quatro fios (`ADC_CONVST`, `ADC_SCLK`, `ADC_DIN`, `ADC_DOUT`). A marca "2308" no chip ao lado do conector confirma o modelo; revisões antigas da DE1-SoC tinham outro conversor (AD7928), e o manual da placa de 2015 descreve esse, não o LTC2308.

As entradas chegam pelo conector J15, de 2×5 pinos. Pelo manual da placa (seção 3.6.12): pino 1 = 5 V, pino 2 = canal 0, pinos 3 a 9 = canais 1 a 7, pino 10 = terra. **Identifique os pinos medindo, e não olhando**: o conector já foi lido girado de 180° no laboratório, e o pino 1 fornece 5 V — ligado por engano ao terra, põe a alimentação em curto. Com o multímetro e a ponta comum na carcaça metálica do conector USB, o pino de 5 V e o de 0 V aparecem sem ambiguidade.

**Faixa de tensão.** Com a referência interna, a faixa de medida é de 0 a 4,095 V no modo de entrada simples (1 mV por passo) e de −2,048 a +2,047 V no modo diferencial, entre dois canais. Entre 4,096 V e cerca de 5,3 V a leitura só satura. **Abaixo de −0,3 V a entrada pode ser danificada**, e não há proteção entre o conector e o chip. Um sinal alternado do gerador de funções precisa de deslocamento DC: por exemplo, 2 V de deslocamento e no máximo 4 Vpp, conferido no osciloscópio *antes* de ligar à placa, com o terra do gerador no pino de terra do conector.

**Sem filtro *anti-aliasing*.** O conector vai direto ao chip, sem nenhum filtro analógico, e a entrada do LTC2308 tem 25 MHz de banda. Qualquer componente acima de $f_s/2$ é amostrado e aparece rebatido no espectro. Quem precisar evitar isso tem de filtrar antes do conector (a folha de dados sugere um RC de 50 Ω e 2000 pF, que limita a banda a 1,6 MHz, ou um passa-baixas mais estreito para a taxa usada). O codec de áudio (seção 3.5), ao contrário, tem filtro.

**Referência de tensão. A placa não tem referência externa.** O LTC2308 usa a própria referência interna, de 2,5 V, que um amplificador dentro do chip leva a 4,096 V; é esse valor que define o fundo de escala e o passo de 1 mV. O chip aceitaria uma referência externa, aplicada

sobre o pino de referência dele, mas na DE1-SoC esses pinos não chegam ao conector J15, que traz só os oito canais, os 5 V e o terra: usar uma referência externa exigiria mexer na placa. Por isso a exatidão absoluta de uma leitura é a da referência interna do chip, e a leitura de 3,3300 V no pino de 3,3 V da placa (seção 3.4.4) ainda precisa ser comparada com um multímetro.

### 3.4.2 Por que um controlador próprio

O projeto original trazia o controlador do *University Program* da Intel para esse conversor, com a instância comentada. Ele converte sem parar, por conta própria, e entrega apenas o último valor lido de cada canal. Para ler uma tensão isso basta; para amostrar um sinal, não: o instante em que cada amostra foi tirada fica incerto em até uma volta inteira da varredura, alguns microssegundos. Essa incerteza vira ruído que cresce com a frequência do sinal — estimado em uns 25 dB de relação sinal-ruído para um tom de 5 kHz, contra os 74 dB que 12 bits permitem.

O `Quartus/adc_captura.v` substitui esse controlador. Um contador conta $D$ ciclos de 50 MHz e, a cada volta, dispara uma conversão (a subida do `CONVST`). O instante da amostra é essa borda, com a precisão do relógio de 20 ns, e a taxa de amostragem é

$$f_s = \frac{50\,\text{MHz}}{D}, \qquad 250 \le D \le 50\,000,$$

de 200 kHz a 1 kHz. Os 50 MHz são o relógio da FPGA, não a taxa de amostragem: quem decide a taxa é o aluno, na janela de aquisição. O cliente converte a $f_s$ pedida no divisor inteiro mais próximo, $D = \text{round}(50\,\text{MHz}/f_s)$ (`divisor_para` em `Python/aquisicao.py`), e o envia na requisição; o servidor o escreve no registrador `adc_divisor` antes de disparar. Como $D$ é inteiro, a taxa real pode diferir um pouco da pedida (44,1 kHz vira 44 091,7 Hz), e é a taxa real que acompanha a captura.

A cada período o controlador:

1. mantém o `CONVST` alto por 96 ciclos (1,92 µs, acima do tempo máximo de conversão);
2. gera 12 pulsos de relógio serial a 12,5 MHz, lendo os 12 bits do resultado e enviando, nos seis primeiros, a palavra de configuração da conversão seguinte (canal e modo);
3. grava a amostra na memória `adc_buf`, de 32 768 palavras.

O primeiro quadro de cada captura é descartado, porque a configuração enviada num quadro vale para a conversão seguinte. A taxa máxima de 200 kHz vem do tamanho do quadro, cerca de 150 ciclos, mais o tempo de aquisição que o conversor exige.

### 3.4.3 Captura única e captura contínua

O controlador tem dois modos, escolhidos pelo servidor:

**Captura única** grava $n \le 32\,768$ amostras na memória e levanta o `done`. O servidor então lê tudo e responde.

**Captura contínua** grava sem parar, usando a memória como *buffer* circular, e mantém um contador (`adc_contador`) de quantas amostras já gravou. O servidor lê o contador, copia as amostras novas e as envia em blocos enquanto a captura segue (seção 5.6). Não há intervalo entre um bloco e outro, porque quem fixa o instante de cada amostra é o contador de período da FPGA, que não para.

Na captura contínua o limite passa a ser o servidor acompanhar: a memória guarda 164 ms de sinal a 200 kHz, e se o servidor atrasar mais que isso as amostras seriam sobrescritas antes de lidas. Isso nunca vira um buraco silencioso no sinal: o servidor relê o contador depois de cada cópia e, se o controlador já passou do ponto, termina a captura com o estado "perdeu". O que foi enviado até ali é contínuo e válido. Medido em 24/09/2026, o HPS acompanha a taxa máxima: 2 milhões de amostras a 200 kHz, em 10 s, sem perda.

Tabela 3.2: Os registradores do controlador do ADC, na ponte leve, e a sua memória.

| Nome | Endereço | Função |
|---|---|---|
| `adc_buf` | `0x20000` (ponte HPS→FPGA) | 32 768 palavras; 12 bits úteis cada |
| `adc_start` | `0xD0` | a subida dispara a captura; descer encerra a contínua |
| `adc_done` | `0xE0` | fim da captura única |
| `adc_config` | `0xF0` | bits 5–0: palavra do LTC2308 (canal, modo); bit 6: captura contínua |
| `adc_divisor` | `0x100` | $D$, de 250 a 50 000 |
| `adc_namostras` | `0x110` | $n$ da captura única |
| `adc_contador` | `0x120` | amostras já gravadas na contínua; 0 em repouso |

### 3.4.4 O que foi validado

Em 24/09/2026, na placa 1, com o bitstream que contém o ADC: as operações que já existiam continuam iguais (testes da FFT, do IIR e dos blocos); a entrada ligada ao terra lê 0,0000 V, com desvio de 0,03 mV; ligada ao 3,3 V da própria placa, 3,3300 V, com desvio de 2,6 mV; a numeração dos canais e o modo diferencial estão certos; e a captura contínua acompanha 200 kHz. Com a entrada solta, a leitura fica perto de 1,9 V: é o próprio conversor carregando o pino, e não uma medida.

Em 07/10/2026, com uma senoide de 1 kHz do gerador de funções no canal 0 (deslocamento de cerca de 2 V, amplitude dentro de 0 a 4 V): o `testa_adc.py` passou em todas as verificações, nas capturas únicas (frequência medida dentro de 0,1 % em cada taxa, sem saturar, ENOB acima de 9) e na captura contínua de 10 s a 200 kHz (2 milhões de amostras, sem perda e sem salto nas emendas entre blocos). Na captura contínua inteira, a frequência medida foi de 1000,013 Hz, a SINAD de 56,1 dB e o ENOB de 9,03 bits. O ENOB mede o conversor *junto com o gerador*; o capítulo 11 comenta a folga.

## 3.5 O codec de áudio

A placa tem um codec de áudio, o WM8731 da Wolfson (hoje Cirrus Logic), com três conectores P2 de 3,5 mm: **LINE-in** (azul), **LINE-out** (verde) e **MIC-in** (rosa). Ele tem um conversor analógico-digital e um digital-analógico estéreo, sigma-delta, com taxas fixas de 8 a 96 kHz. Desde 13/10/2026 o Morphe o usa, na branch `estagio/v1.8-codec`. É a única saída analógica da plataforma: com ele a FPGA também *gera* som.

Tabela 3.3: Os dois conversores da placa.

| | LTC2308 (conector J15) | WM8731 (codec) |
|---|---|---|
| tipo | aproximações sucessivas, 12 bits, 8 canais | sigma-delta, 16 bits no Morphe, 2 canais |
| acoplamento | DC, de 0 a 4,095 V | AC: não mede tensão contínua |
| taxa | qualquer uma até 200 kHz no Morphe | 48 828 Hz no Morphe |
| *anti-aliasing* | **não tem** | filtro digital de dizimação |
| saída analógica | não tem | DAC e LINE-out |

### 3.5.1 Configuração pelo HPS

O WM8731 é configurado por $I^2C$, escrevendo registradores de 9 bits no endereço `0x1A`. O barramento passa por um multiplexador da placa: com o pino `HPS_I2C_CONTROL` (GPIO48 do HPS) em 1, quem fala com o codec é o controlador I2C0 do HPS. O programa `C/codec.c` faz isso pelo Linux da placa, sem mudar o bitstream:

```
make codec
./codec digital                          LINE-in (azul) -> FPGA -> LINE-out
./codec --entrada mic digital            MIC-in (rosa), com +20 dB
./codec --volume -20 --entrada mic digital
./codec bypass                           LINE-in -> LINE-out, só no chip
./codec desliga
```

O `--volume` vai de $-73$ a $+6$ dB (padrão $-10$). A imagem da placa não tem `/dev/i2c`, e o programa então programa o controlador I2C0 direto pelos registradores. Um detalhe custou uma tarde: o I2C0 do Cyclone V só gera a condição de STOP se o último byte levar o bit 9 do `IC_DATA_CMD`; sem ele, o barramento fica preso.

### 3.5.2 O áudio na FPGA

O `audio_i2s.v` troca as amostras com o codec pelo formato I2S, palavras de 16 bits em complemento de dois. O codec é o *mestre* dos relógios de áudio; a FPGA só lhe entrega o relógio-mestre, $50\,\text{MHz}/4 = 12{,}5\,\text{MHz}$, de um contador, sem PLL. Com o codec em $256 f_s$,

$$f_s = \frac{12{,}5\,\text{MHz}}{256} = 48\,828{,}125\ \text{Hz},$$

exata, e não 48 kHz: para 48 kHz seria preciso um PLL fracionário, e uma taxa exata conhecida basta no laboratório. Tudo roda no domínio de 50 MHz, como o resto do projeto.

A chave **SW9** escolhe o que vai para o DAC:

**SW9 baixa** o que o `audio_buf.v` mandar, comandado pelo HPS: silêncio, a reprodução de um sinal vindo do computador ou o *loopback* (seção 3.5.3). Até 14/10/2026 o *loopback* ficava sempre ligado aqui.

**SW9 alta** uma onda quadrada de $\pm 4096$ ($-18$ dBFS) gerada na FPGA, a $48\,828/48 = 1017$ Hz. Prova o DAC sem nenhuma fonte de áudio, e tem prioridade sobre tudo.

O `audio_vu.v` mostra o nível do que chega nos displays HEX0 a HEX4, em escala logarítmica, 6 dB por segmento. Assim a recepção pode ser vista sem cabo nenhum: um fone ligado no MIC-in funciona como microfone.

***Anti-aliasing.*** O conversor do codec amostra muito acima de $f_s$ e reduz a taxa com um filtro digital de dizimação. Na folha de dados (tabela 31, modo normal), a banda passante vai até $0{,}4535 f_s$ (22,1 kHz, $\pm 0{,}05$ dB), a atenuação é de 6 dB em $f_s/2$ e de pelo menos 60 dB a partir de $0{,}5465 f_s$ (26,7 kHz). Entre 24,4 e 26,7 kHz o rebatimento é atenuado só em parte. Há também um passa-altas de 3,7 Hz na entrada, que tira o nível contínuo.

### 3.5.3 O áudio entre o codec e o computador

O `audio_buf.v` leva o áudio do codec para a memória que o HPS lê, e de volta. Ele usa a **mesma memória e os mesmos registradores do ADC** (`adc_buf`, 32 768 palavras): não sobra registrador livre no Platform Designer, e acrescentar um exigiria regenerar o sistema. Quem escolhe é o bit 31 do `adc_divisor`, que o ADC nunca usa (o divisor dele vai no máximo a 50 000): com ele em 1, o `ghrd_top.v` liga a memória, o `start`, o `done` e o contador ao áudio, e o `adc_captura.v` não vê o `start`. Cada palavra guarda um quadro estéreo: `{direito[31:16], esquerdo[15:0]}`.

As operações, em `adc_config[1:0]`, com o bit 6 escolhendo o modo contínuo (a memória como *buffer* circular):

**0, captura** entrada do codec → memória;

**1, reprodução** memória → saída do codec;

**2, captura com *loopback*** as duas coisas ao mesmo tempo;

**3, *loopback*** entrada → saída, sem memória.

O ritmo é o do próprio codec: ele é o mestre dos relógios, e cada quadro recebido é também um quadro enviado. O servidor atende a operação `AUDIO` (capítulo 5): a captura chega ao cliente em blocos, sem limite de duração; a reprodução de um sinal maior que a memória (0,67 s) toca enquanto o servidor a reabastece atrás do contador; o *loopback* fica ligado entre conexões, e qualquer operação do ADC ou do áudio o pausa e religa sozinha. No aplicativo, é a janela **Áudio** (seção 7.2).

### 3.5.4 O que foi validado

Na placa 1, com um bitstream de teste compilado no Quartus 23.1:

- 13/10/2026: o codec responde no I2C; o tom de 1017 Hz sai na LINE-out; com um fone no MIC-in, o medidor fica em 5 segmentos em repouso e chega a 8 mexendo no fone, e a LINE-in vazia fica em 1;
- 15/10/2026: o *loopback* é audível. Com o microfone de um fone no MIC-in, uma caixa de som na LINE-out e `./codec --volume -20 --entrada mic digital`, o sopro no microfone sai na caixa, passando pela FPGA;
- 15/10/2026, com o `audio_buf.v`: captura de 10 blocos de 1024 quadros e de 3 s seguidos (4,5 vezes a memória) sem perda; reprodução de 0,5 s e de 4 s (195 312 quadros, seis vezes a memória) sem falha, com o tom ouvido na caixa; o ADC continua funcionando com o *loopback* ligado; `morphe_ping` 4/4. A janela Áudio foi acionada contra a placa (captura, gerador, reprodução da captura, os três *loopbacks*).

O `Quartus/testbench/tb_audio_i2s.v` simula o `audio_i2s.v` contra um modelo do WM8731 em modo mestre (recepção, *loopback* e tom); o `tb_audio_buf.v`, as quatro operações do `audio_buf.v`, únicas e contínuas, com a memória dando a volta; e o `tb_audio_vu.v`, o medidor. Na placa, `Python/ferramentas/testa_codec.py`.

**Cuidado com a microfonia**: com microfone e caixa de som no mesmo ambiente, o microfone capta a própria caixa e a realimentação apita. Comece com o volume baixo e afaste os dois.

O codec precisa ser configurado na placa (`./codec digital`) depois de cada boot; sem isso não há relógio de áudio, e o servidor responde à janela Áudio com uma mensagem que diz o que fazer. Desde 15/10/2026 quem faz isso é o `morphe-up.sh`, a cada execução, com o microfone como entrada e volume de −20 dB (`--codec linha` para a LINE-in): a configuração vive no codec e sobrevive a reprogramar a FPGA, e depois de um boot o `morphe-up.sh` tem de rodar de qualquer jeito.

# Capítulo 4

# O servidor na placa

O servidor é um programa em C, `C/morphe_server.c`, que roda no HPS de cada placa. Ele é a única parte do sistema que fala com a FPGA, e faz pouco: recebe uma requisição pela rede, confere, copia os dados para as memórias do acelerador, dispara, espera e devolve o resultado. Este capítulo descreve como ele faz isso e por que algumas das suas regras existem.

## 4.1 Estrutura

O programa tem um único fio de execução e atende **uma conexão por vez**. O laço principal é o mais simples possível:

1. abre o `/dev/mem` e mapeia as duas pontes (seção 3.2);
2. abre um *socket* TCP na porta 5000 (ou na que vier como argumento) com uma fila de espera de quatro conexões, `listen(srv, 4)`;
3. repete para sempre: `accept` → atende → `close`.

Atender uma conexão é ler o cabeçalho de 20 bytes, conferir a assinatura e a versão, e despachar pelo campo `opcode` para uma função por operação: `handle_conv`, `handle_fir`, `handle_fft`, `handle_ifft`, `handle_iir`, `handle_adc`, `handle_adc_continuo` e `handle_ping`. Um opcode desconhecido recebe o status `BAD_OPCODE`.

As operações não correm em paralelo: não há *fork* nem fila própria. Quem espera é o sistema operacional: enquanto uma operação está sendo atendida, as conexões seguintes ficam na fila do `listen`. Essa escolha é do projeto original e foi mantida porque é correta: a FPGA tem um acelerador de cada tipo, e duas operações ao mesmo tempo no mesmo acelerador seriam um erro. O que ela custa em latência quando vários alunos usam a mesma placa está medido no capítulo 8.

A única outra *thread* é a da **porta de estado** (UDP, porta 5001, desde 28/09/2026). O laço principal anota, ao começar e ao terminar cada operação, qual é, desde quando e de qual computador; a *thread* responde isso a quem perguntar, na hora, mesmo com a placa ocupada. Ela não toca a FPGA — só essas anotações, protegidas por um *mutex*, e a marca de FPGA preparada — e, se a porta não abrir, o servidor segue sem ela. O capítulo 8 (seção 8.2) descreve o que ela responde e quem usa.

### Uma função por operação, todas com o mesmo roteiro

Cada `handle_*` segue a mesma sequência, na mesma ordem:

1. confere se a FPGA foi preparada (seção 4.3); se não, recusa sem tocar em nada;

2. confere os tamanhos contra os limites de `morphe_config.h` e o tipo dos dados;
3. recebe o *payload* inteiro para um *buffer* local;
4. escreve nas memórias de entrada, **completando com zeros** até o tamanho do hardware;
5. leva o `start` a 0, espera 1 µs, leva a 1;
6. espera o `done` (seção 4.2);
7. devolve o `start` a 0, lê as saídas e envia a resposta.

O enchimento com zeros do passo 4 não é detalhe: os aceleradores sempre processam o bloco inteiro de 1024 amostras, e o que tivesse sobrado na memória de uma operação anterior entraria na conta. No IIR isso apareceria como uma cauda na saída; na convolução, como amostras erradas no fim.

## 4.2 Esperar a FPGA

O servidor não usa interrupções. Depois do `start`, lê o registrador `done` a cada 50 µs até ele subir. Se não subir em **5 segundos**, a operação é abandonada, o `start` volta a 0 e o cliente recebe o status `FPGA_TIMEOUT`.

Nenhuma operação normal chega perto desse prazo: a mais longa, a convolução de 1024 por 1024 amostras, leva cerca de 170 ms na FPGA. Um tempo esgotado quer dizer que o acelerador não está funcionando, e na prática quase sempre tem uma de duas causas: o *tether* da licença caiu e o bitstream parou (seção 9.3), ou o servidor foi compilado com um `hps_0.h` que não corresponde ao bitstream carregado.

A exceção são as capturas do ADC, cuja duração depende do pedido — 32 768 amostras a 1 kHz levam 33 s. Nelas o prazo é calculado da própria captura: a duração esperada mais 2 s.

Cada operação registra no *log* os seus tempos em três fases — `entrada` (copiar para a FPGA), `fpga` (do `start` ao `done`) e `saida` (ler e enviar). O número do meio é o que interessa para diagnóstico: ele mede o hardware sozinho, sem a rede.

## 4.3 A marca de FPGA preparada

Esta é a regra mais importante do servidor, e a que custou mais caro para descobrir.

Quando uma DE1-SoC liga, o carregador de boot (U-Boot) programa a FPGA com o bitstream de fábrica gravado no cartão SD, o `soc_system.rbf`. Nesse bitstream os registradores e memórias do Morphe **não existem**. No Cyclone V, um acesso a um endereço sem escravo na ponte HPS→FPGA não devolve erro: **trava o barramento do processador inteiro**, sem prazo. A placa some da rede, e nem o console serial responde. Foi o que aconteceu com a placa 2 do laboratório a cada boot, entre 17 e 21/09/2026: o servidor subia sozinho no boot e zerava os registradores de `start` antes de a FPGA ter o bitstream do Morphe.

Desde então o servidor segue esta regra:

> **Nenhum acesso à FPGA enquanto não existir o arquivo `/var/run/morphe-fpga-preparada`.**

Quem cria esse arquivo é o `morphe-up.sh`, por SSH, logo depois de o programador do Quartus confirmar que gravou o bitstream do Morphe (capítulo 9). O `/var/run` é uma pasta em memória: o arquivo some sozinho no próximo boot, que é exatamente quando o U-Boot devolve a FPGA ao bitstream de fábrica. A marca e o bitstream que ela atesta desaparecem juntos.

Sem a marca, o servidor sobe normalmente, responde ao `PING` com `fpga_preparada=0` e recusa toda operação com o status `FPGA_NAO_PREPARADA` e a mensagem "rode ./morphe-up.sh nesta placa". Na primeira requisição depois que a marca aparece, e só então, ele zera os registradores de `start` de todos os aceleradores.

### A segunda proteção: o `sysid`

O ADC trouxe uma variação do mesmo problema. Um servidor compilado com os endereços do ADC pode encontrar na FPGA um bitstream mais antigo, sem ele — por exemplo, se o `morphe-up.sh` ainda não rodou depois de uma atualização. Os registradores do ADC não existiriam, e tocá-los travaria o barramento do mesmo jeito.

Todo bitstream do Morphe tem um componente `sysid` no mesmo endereço, com a data e hora em que o sistema foi gerado no Platform Designer. O `gen_hps_header.py` copia esse valor para o `hps_0.h` (`SYSID_QSYS_TIMESTAMP`). Antes de tocar em qualquer registrador do ADC, o servidor lê o `sysid` da FPGA e o compara com o do cabeçalho; se forem diferentes, recusa com a mensagem "o bitstream na FPGA não é o deste servidor". O `PING` anuncia `adc_n_max=0` sempre que o ADC não pode ser usado.

Há ainda uma terceira camada, em tempo de compilação: o código do ADC só é compilado se o `hps_0.h` tiver a memória `adc_buf` (`MORPHE_TEM_ADC`). Um cabeçalho de antes do ADC produz um servidor que responde a essas operações com `BAD_OPCODE` e uma explicação.

## 4.4 Outros cuidados

**Respostas de erro que chegam ao cliente.** Quando o servidor recusa uma requisição, o cliente pode já ter enviado parte do *payload*. Fechar um *socket* com dados não lidos faz o Linux enviar um *reset* em vez de um fechamento normal, e o *reset* descarta a resposta que acabou de ser enviada: o cliente via "*connection reset by peer*" no lugar da mensagem de erro. Por isso, depois de toda resposta de erro, o servidor lê e descarta o que houver na entrada (`drain_pending`), com um limite de tempo e de tamanho.

**Cliente que fecha a conexão no meio.** Escrever num *socket* cujo outro lado fechou gera o sinal `SIGPIPE`, cuja ação padrão encerra o processo. Até 24/09/2026 um aluno que fechasse a janela durante uma operação longa derrubava o servidor da placa para todos. Agora o sinal é ignorado e o envio só devolve erro.

**Pacotes de depuração.** A cada operação o servidor grava, no seu diretório na placa, um pacote `.mrph` com a entrada e a saída (seção 7.6). Para não encher o cartão SD em silêncio, ele guarda os 100 mais recentes de cada tipo de operação e apaga os mais antigos. A variável de ambiente `MORPHE_DUMP_MAX` muda esse número; 0 desliga a gravação.

**Modo de diagnóstico da FFT.** Com `MORPHE_FFT_INVERSE=1` no ambiente, a operação FFT roda com o bit `inverse` ligado. Serviu para provar, em 10/09/2026, que esse bit estava vivo no hardware antes de existir a operação IFFT; não é usado na operação normal.

## 4.5 Compilação e início automático

O servidor é compilado **na própria placa**, com o `gcc` do Linux dela, pelo `C/Makefile`. As opções ativam as otimizações para o Cortex-A9 (`-O3`, `-mcpu=cortex-a9`, `-mfpu=neon`). Os seis arquivos necessários — `morphe_server.c`, `morphe_protocol.h`, `morphe_config.h`, `hps_0.h`,

`address_map_arm.h` e o `Makefile` — são enviados e compilados pelo `morphe-up.sh`, que recompila com `make -B` sempre que algum deles mudou. Compilar à mão não é necessário, e o servidor precisa rodar como `root` por causa do `/dev/mem`.

**No boot.** O `C/autostart/instala-autostart.sh`, rodado uma vez em cada placa, registra o servidor como serviço do sistema (*systemd* ou *init.d*, o que a imagem do cartão tiver), com reinício automático se ele cair. Com o serviço instalado, a placa aparece na rede para os clientes assim que liga, recusando operações até ser preparada. Só o servidor de 21/09/2026 em diante pode ir para o início automático: o anterior tocava a FPGA ao subir.

## 4.6 Códigos de status

Toda resposta traz um status. Quando ele não é zero, o *payload* é uma mensagem em texto, que o cliente mostra ao aluno.

Tabela 4.1: Códigos de status das respostas (`C/morphe_protocol.h`).

| **Código** | **Nome** | **Quando aparece** |
|---|---|---|
| 0 | `OK` | a operação foi feita |
| 1 | `BAD_MAGIC` | o cabeçalho não começa com `MRPM`: não é um cliente Morphe |
| 2 | `BAD_VERSION` | versão do protocolo diferente de 1 |
| 3 | `BAD_OPCODE` | operação desconhecida, ou ADC pedido a um servidor compilado sem ele |
| 4 | `BAD_DTYPE` | tipo de dado que a operação não aceita (a FFT, a IFFT e o ADC só aceitam `int32`) |
| 5 | `BAD_SIZE` | tamanho fora dos limites do hardware, número de seções do IIR, taxa de amostragem ou configuração do ADC inválidos |
| 6 | `FPGA_TIMEOUT` | o acelerador não levantou o `done` no prazo (seção 4.2) |
| 7 | `INTERNAL_ERROR` | só no FIR, se o registrador `fir_error` estiver em 1; como nada no bitstream atual aciona esse registrador (seção 3.3.2), na prática não aparece |
| 8 | `FPGA_NAO_PREPARADA` | a placa não foi preparada pelo `morphe-up.sh`, ou o bitstream carregado não é o do servidor |

# Capítulo 5

# O protocolo Morphe-TCP

Cliente e servidor conversam por um protocolo binário próprio, sobre TCP, na porta 5000. Ele é definido em dois arquivos que precisam concordar byte a byte: `C/morphe_protocol.h`, do lado do servidor, e `Python/morphe_protocol.py`, do lado do cliente.

## 5.1 Uma conexão por operação

Cada operação abre a sua própria conexão, e a conexão acaba com ela:

1. o cliente conecta;
2. envia um cabeçalho de requisição de 20 bytes e, logo em seguida, o *payload* com os dados;
3. o servidor responde com um cabeçalho de resposta de 20 bytes e o *payload* do resultado (ou uma mensagem de erro);
4. o servidor fecha a conexão.

Não existe sessão, autenticação nem estado guardado entre conexões. Isso é o que permite ao cliente escolher uma placa diferente a cada operação e a vários clientes intercalarem operações na mesma placa (capítulo 8). A única exceção ao roteiro é a captura contínua do ADC, em que a resposta é uma sequência de blocos enquanto a captura dura (seção 5.6).

## 5.2 Os cabeçalhos

Todos os campos são inteiros sem sinal em ordem de rede (*big-endian*). A Figura 5.1 mostra os dois cabeçalhos.

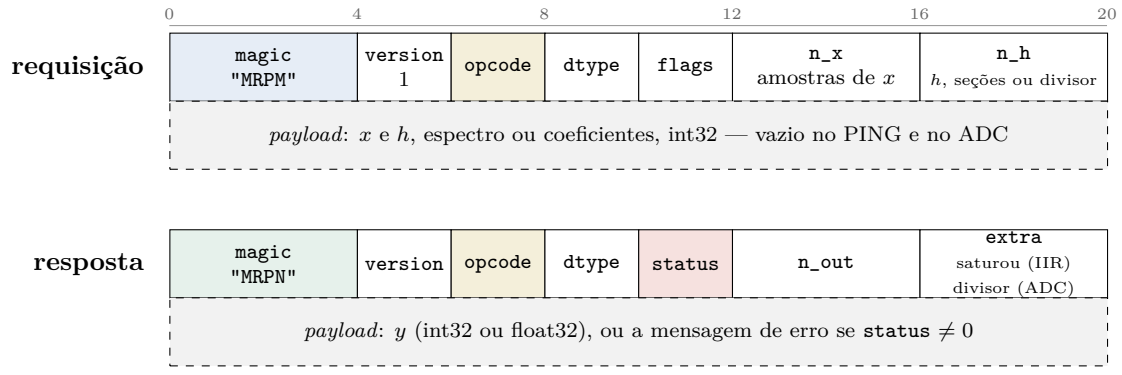



Figura 5.1: Os cabeçalhos de requisição e de resposta, em bytes.

`magic` a assinatura: `0x4D52504D` ("MRPM") na requisição e `0x4D52504E` ("MRPN") na resposta. Um servidor que recebe outra coisa responde `BAD_MAGIC` sem ler mais nada.

`version` sempre 1.

`opcode` a operação (Tabela 5.1); a resposta repete o da requisição.

`dtype` o tipo dos números no *payload*: 1 para `int32`, 2 para `float32`. Na prática o cliente sempre envia `int32` já em ponto fixo; o `float32` existe na convolução e no FIR por compatibilidade com o projeto original.

`flags` 0, exceto no ADC, em que carrega a palavra de configuração do conversor; no IIR, 1 pede para o bloco partir de um estado dado e devolver o estado final (seção 5.5).

`n_x`, `n_h` os tamanhos. O significado muda com a operação: no IIR, `n_h` é o número de seções; no ADC, o divisor do relógio que dá a taxa de amostragem.

`status` 0 quando deu certo; os outros valores estão na Tabela 4.1. Com status diferente de zero, o *payload* é uma mensagem de texto de `n_out` bytes.

`n_out` quantos valores (ou bytes de mensagem) vêm no *payload* da resposta.

`extra` uma palavra livre para a operação: no IIR, bit 0 = alguma amostra saturou e bit 1 = o estado final vem depois de $y$; o divisor usado no ADC; 0 nas outras.

## 5.3 As operações

Algumas observações sobre a tabela.

**A FFT e a IFFT recusam tamanhos diferentes de 1024.** A convolução completa com zeros o que faltar, porque acrescentar zeros ao fim de um sinal não muda a convolução. Com um espectro isso não vale: completar $X[k]$ com zeros muda o sinal que ele representa. Por isso as duas exigem exatamente 1024 pontos, e sinais de outros tamanhos são tratados no cliente (seção 7.7).

**A FFT devolve números em ponto flutuante.** É a única operação em que o servidor converte: ele conhece o expoente de bloco que o núcleo da FFT calculou (seção 3.3.3) e o aplica, junto com o fator $2^{-8}$ do Q15.8, antes de enviar. O cliente ainda desfaz a escala que ele mesmo aplicou na entrada (seção 6.3).

**A IFFT recebe o dobro de dados.** Um espectro é complexo por natureza, então o *payload* tem duas palavras por ponto. O servidor satura cada uma na faixa do Q15.8 antes de escrever na FPGA.

**A palavra de configuração do ADC.** São os seis bits que o LTC2308 recebe a cada conversão, na ordem da folha de dados: `S/D`, `O/S`, `S1`, `S0`, `UNI`, `SLP`. Os quatro primeiros escolhem o canal e se a entrada é simples (contra o terra) ou diferencial (entre dois canais); `UNI` escolhe unipolar (0 a 4,095 V) ou bipolar (±2,048 V). O modo de economia de energia (`SLP`) é recusado, porque a referência interna leva 200 ms para voltar dele. Quem monta a palavra é o `Python/aquisicao.py`; a janela só oferece o canal e o modo.

**O áudio vai em palavras de 32 bits.** Cada quadro estéreo é uma palavra `{direito, esquerdo}`, 16 bits cada, em complemento de dois, como está na memória da FPGA. O `Python/audio.py` separa e junta os canais.

Tabela 5.1: As operações do protocolo e o que vai em cada *payload*. Todos os valores ocupam 4 bytes.

| | **Operação** | **Requisição** | **Resposta** |
|---|---|---|---|
| 1 | `CONV` | $n_x$ amostras de $x$, depois $n_h$ de $h$, Q15.16; $1 \le n_x, n_h \le 1024$ | $n_x + n_h - 1$ amostras de $y$, Q15.16 |
| 2 | `FFT` | 1024 amostras reais em Q15.8 ($n_x = 1024$) | 1024 valores complexos em `float32`, parte real e imaginária intercaladas, já em escala real |
| 3 | `PING` | nada | texto `chave=valor`, uma por linha (seção 5.4) |
| 4 | `FIR` | como a `CONV`: $x$ e os coeficientes $h$ | como a `CONV` |
| 5 | `IFFT` | 1024 valores complexos em Q15.8, real e imaginário intercalados ($2 \times 1024$ palavras) | como a `FFT`, sem o fator $1/N$ |
| 6 | `IIR` | $n_x \le 1024$ amostras em Q15.16, depois $5 \cdot n_h$ coeficientes Q15.16 na ordem $b_0, b_1, b_2, a_1, a_2$ de cada seção; $1 \le n_h \le 16$; com `flags` = 1, mais $4 \cdot n_h$ palavras do estado inicial | $n_x$ amostras Q15.16 (com `flags` = 1, mais o estado final); `extra`: bit 0 = saturou, bit 1 = estado |
| 7 | `ADC` | nada; $n_x$ = amostras (até 32 768), $n_h$ = divisor, `flags` = configuração | $n_x$ códigos do conversor, `int32`; `extra` = divisor |
| 8 | `ADC_CONTINUO` | como o `ADC`, com $n_x$ sem limite (0 = até o cliente parar) | cabeçalho com $n_{out} = 0$ e depois blocos (seção 5.6) |
| 9 | `AUDIO` | `flags` = 0, captura: $n_x$ quadros (0 = até parar), $n_h = 1$ com *loopback*; 1, reprodução: $n_x$ quadros seguem o cabeçalho; 2, *loopback*: $n_h = 1$ liga, 0 desliga; 4, ao vivo: $n_h$ = seções, seguem o aquecimento e os $5n_h$ coeficientes | captura: blocos, como o `ADC_CONTINUO`, com `extra` = 1024 ($f_s = 50\,\text{MHz}/1024$); reprodução: `extra` = como terminou; ao vivo: blocos com {saída, entrada} e o tempo de placa no estado |

## 5.4 A descoberta: `PING`

O `PING` é só um cabeçalho, e a resposta é um texto curto que descreve a placa e o hardware que ela tem. Um exemplo, de uma placa preparada:

```
service=morphe
version=1
hostname=de1soc-02
fft_n=1024
conv_n_max=1024
iir_secoes_max=16
adc_n_max=32768
adc_fs_max=200000
uptime_s=5412
fpga_preparada=1
```

(a resposta completa traz mais alguns campos, como os bits fracionários de cada formato.) O cliente usa o `PING` para achar placas na rede e para conferir que uma placa está preparada (`fpga_preparada`); placas não preparadas não são escolhidas. O campo `adc_n_max` diz se o ADC pode ser usado naquela placa agora (0 quando não pode). O tempo que a resposta leva também é usado, para escolher a placa menos ocupada (capítulo 8).

## 5.5 O estado do IIR entre execuções

O bloco IIR processa 1024 amostras por execução, e um IIR tem memória: para filtrar um sinal maior em blocos, cada bloco precisa partir do estado em que o anterior terminou. Na Forma Direta I esse estado são quatro números por seção, $x[n-1]$, $x[n-2]$, $y[n-1]$ e $y[n-2]$. Com `flags` $= 1$, o pedido traz, depois dos coeficientes, o estado inicial ($4 \cdot n_h$ palavras Q15.16, nessa ordem, seção por seção), e a resposta traz, depois de $y$, o estado final, o de depois das 1024 amostras do bloco. Passar o estado final de um bloco como inicial do seguinte dá a mesma saída, bit a bit, que filtrar o sinal inteiro de uma vez, sem nenhuma amostra de aquecimento.

O estado viaja pela SRAM dos coeficientes, que tem 256 palavras e só usa 80: a palavra 120 é o modo (1 = carregar o estado inicial), a 121 é uma marca que o bloco escreve ao terminar, as 128 a 191 são o estado inicial e as 192 a 255, o final. Um bitstream anterior a 19/10/2026 não escreve a marca; o servidor percebe, responde só com $y$ e deixa o bit 1 do `extra` em zero. Se o estado inicial pedido não era zero, a resposta é um erro, porque esse bitstream o teria ignorado. O estado vai sempre em `int32`: em `float32` perderia bits.

## 5.6 A captura contínua do ADC

A captura contínua é a única operação cuja resposta não cabe numa mensagem só: ela pode durar minutos, e o sinal é enviado enquanto é capturado. Depois do cabeçalho de resposta (com `status` 0, $n_{out} = 0$ e o divisor em `extra`), vem uma sequência de blocos, cada um com o formato

$$\underbrace{n}_{\texttt{uint32}}\ \underbrace{estado}_{\texttt{uint32}}\ \underbrace{c_0\, c_1 \ldots c_{n-1}}_{n\times\texttt{int32}}$$

O *estado* de cada bloco é 0 enquanto a captura segue. O último bloco tem $n = 0$ e um estado que diz por que ela terminou:

| Estado | Significado |
|---|---|
| 1 | fim normal: chegou ao total pedido, ou o cliente pediu para parar |
| 2 | o servidor não acompanhou e amostras foram perdidas; o que veio antes vale |
| 3 | o hardware parou de produzir amostras |

Os blocos são contínuos no tempo: a última amostra de um e a primeira do seguinte estão separadas por exatamente um período de amostragem. Para parar a captura antes do total, o cliente envia um byte qualquer pela mesma conexão (é o que o botão *Parar* faz), ou simplesmente a fecha. Como funciona do lado do hardware está na seção 3.4.

## 5.7 Como acrescentar uma operação

Uma operação nova toca sempre os mesmos lugares, e esquecer qualquer um deles produz um erro que não aponta para a causa:

1. `C/morphe_protocol.h`: o número da operação (`MORPHE_OP_*`) e, no comentário do topo, o formato do *payload*;
2. `C/morphe_server.c`: a função `handle_*`, seguindo o roteiro do capítulo 4 — a primeira linha confere a marca de FPGA preparada — e o seu `case` no `serve_connection`;
3. `Python/morphe_protocol.py`: a constante `OP_*`, a função `build_*_request` e a `decode_*_response`;
4. `C/morphe_config.h` e `Python/morphe_config.py`, juntos, se a operação tiver limites próprios;
5. se a operação usa hardware novo: o Platform Designer, o `hps_0.h` regenerado e um bitstream novo (capítulo 10).

Mudar o formato de uma operação que já existe, em vez de criar outra, quebra os clientes instalados nos PCs até que sejam atualizados. Se for inevitável, o lugar de sinalizar é o campo `version`.

# Capítulo 6

# Representação numérica

Tudo o que a FPGA calcula é em ponto fixo, e a diferença entre esse cálculo e o do computador é o que o Morphe existe para mostrar. Este capítulo reúne o que se sabe sobre essa diferença: os formatos usados, a regra que governa o erro, o que o núcleo da FFT faz por dentro e os números medidos na placa. A versão com todas as medições está em `docs/PRECISAO-NUMERICA.md`.

## 6.1 Os dois formatos

O Morphe usa **dois** formatos de ponto fixo, um para cada família de aceleradores (Figura 6.1). Os dois têm a mesma faixa, mas passos muito diferentes, e confundi-los é o erro mais comum de quem mexe no código.

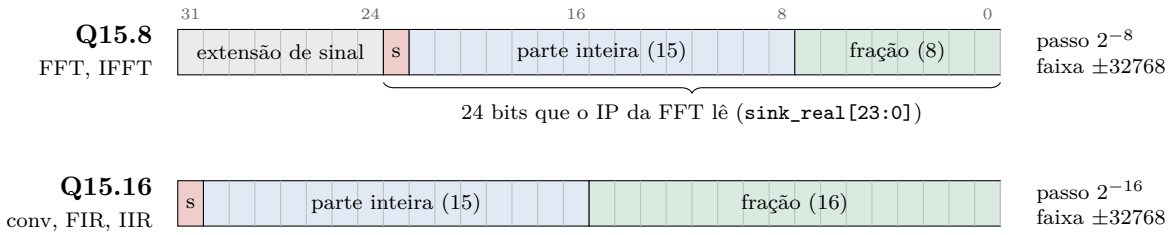



Figura 6.1: Os dois formatos de ponto fixo, bit a bit.

Tabela 6.1: Os formatos de ponto fixo e onde cada um é usado.

| Formato | Bits | Passo $\Delta$ | Faixa | Usado em | Por quê |
|---|---|---|---|---|---|
| Q15.8 | 24 | $2^{-8} \approx 3{,}9 \times 10^{-3}$ | ±32 768 | FFT, IFFT | o núcleo da Intel lê 24 bits |
| Q15.16 | 32 | $2^{-16} \approx 1{,}5 \times 10^{-5}$ | ±32 768 | conv., FIR, IIR | multiplicadores próprios, de 32 bits |

A codificação é a mesma para os dois, feita no cliente:

$$q = \mathrm{round}\Big(v \cdot 2^{f}\Big),$$

com $f$ o número de bits fracionários, saturando na faixa do formato. No Q15.8 o valor ocupa os 24 bits de baixo de uma palavra de 32, com o bit de sinal repetido nos 8 de cima para que o HPS leia o número certo; o núcleo da FFT ignora esses 8 bits. As funções estão em `Python/dsp_core.py` (`fft_q1508_encode` e `float_to_q1516`).

## 6.2 A regra dos 6 dB por bit

O arredondamento comete um erro distribuído uniformemente entre $-\Delta/2$ e $+\Delta/2$, com variância $\Delta^2/12$. Se o sinal for multiplicado por um fator $s$ antes de codificar e dividido por $s$ depois, o passo efetivo passa a ser $\Delta/s$, e a relação sinal-ruído melhora em

$$20 \log_{10} s \text{ dB}, \qquad \text{ou seja, } 6{,}02\,\text{dB a cada vez que } s \text{ dobra.}$$

Numa operação linear — a FFT, a IFFT, a convolução — essa escala se desfaz exatamente na saída, então o ganho é de graça. O quanto se pode ganhar depende da faixa que o sinal deixa sem uso:

$$\text{ganho máximo} = 6{,}02 \cdot \log_2 \frac{32\,768}{\max |v|} \text{ dB.}$$

Um sinal de amplitude 3 codificado numa faixa que vai a 32 768 desperdiça cerca de 13 bits.

**Medido na placa.** A Tabela 6.2 mostra a mesma FFT de um sinal de pico 3,3 feita com escalas crescentes, comparada com a do NumPy. De 1 a 64 são 6 bits: a teoria prevê 36,1 dB de ganho, e a placa deu 36,2 dB.

Tabela 6.2: FFT de um sinal de pico 3,3 com escalas crescentes, comparada com `np.fft.fft` (placa, 10/09/2026).

| **Escala** | **Pico de $x$** | **SNR** | **Ganho** |
|---:|---:|---:|---:|
| 1 | 3,3 | 50,42 dB | — |
| 4 | 13,2 | 62,74 dB | +12,32 dB |
| 16 | 52,8 | 75,07 dB | +24,65 dB |
| 64 | 211,1 | 86,64 dB | +36,22 dB |
| 256 | 844,3 | 91,74 dB | +41,33 dB |
| 1024 | 3 377,3 | 91,66 dB | +41,24 dB |
| 4096 | 13 509,1 | 91,55 dB | +41,13 dB |

Acima da escala 256 a melhora para, em cerca de **92 dB**. A partir daí o limite deixa de ser o Q15.8 da entrada e passa a ser a aritmética interna do núcleo da FFT, explicada na seção 6.4.

## 6.3 A normalização da entrada

Por causa dessa regra, o cliente **normaliza** a entrada da FFT e da IFFT por padrão: multiplica o sinal pelo fator que o leva a 98 % da faixa do Q15.8, envia, e divide o resultado pelo mesmo fator. As janelas têm uma caixa "Normalizar entrada", ligada, que permite desligar e ver a diferença.

Tabela 6.3: Efeito da normalização, medido na placa (10/09/2026).

| **Operação** | **Sem normalizar** | **Normalizada** | **Escala usada** |
|---|---:|---:|---:|
| FFT, sinal de pico 3,3 | 50,42 dB | 91,53 dB | 9 737 |
| IFFT, espectro de pico 1280 | 91,17 dB | 92,69 dB | 25,09 |

A IFFT ganha pouco *neste* espectro porque ele já ocupava bem a faixa. O caso em que ela é decisiva é o contrário, e ele é comum. O espectro de um sinal pode ser até $N$ vezes maior que o sinal:

$$|X[k]| \le \sum_n |x[n]| \le N \cdot \max |x[n]|,$$

ou seja, até 60 dB acima, com $N = 1024$. Um espectro assim satura o Q15.8 nos picos; um espectro pequeno tem os seus pontos menores abaixo de $\Delta/2 = 1/512$, e eles viram exatamente zero. Normalizado, um espectro $10^4$ vezes menor volta com erro de $5{,}9 \times 10^{-8}$, contra $2{,}7 \times 10^{-3}$ sem normalizar.

## 6.4 O que o núcleo da FFT faz por dentro

O núcleo *FFT II* da Intel está configurado com entrada de 24 bits, fatores de giro de 18 bits e *ponto flutuante em bloco*. A FFT de 1024 pontos tem cinco estágios de base 4 ($1024 = 4^5$). A cada estágio o núcleo verifica se os valores cresceram e, se preciso, desloca **as 1024 amostras juntas**, acumulando um expoente único para o bloco:

$$y = y_{\text{bruto}} \cdot 2^{-e}.$$

O adaptador captura $e$ e o servidor o aplica (seção 3.3.3). Duas consequências:

1. o ponto flutuante em bloco **não recupera** o que se perdeu na entrada. Um valor que ficou abaixo de 1/512 já era zero antes de o núcleo começar. Por isso a normalização continua valendo 6 dB por bit;
2. o expoente é **compartilhado**. Os pontos do espectro muito abaixo do pico carregam menos bits efetivos, porque o deslocamento foi decidido pelo maior deles. Isso, junto com os fatores de giro de 18 bits, é o que põe o teto em cerca de 92 dB.

A normalização do cliente e o ponto flutuante em bloco resolvem problemas diferentes: a primeira posiciona o sinal *antes* de ele virar 24 bits; o segundo evita que ele estoure *durante* a transformada. Um não substitui o outro.

**A IFFT não aplica o $1/N$.** O núcleo entrega $N$ vezes a IDFT matemática. Isso foi medido, não suposto: enviando um sinal real pelo caminho da FFT com o bit `inverse` ligado, a magnitude do que volta é a da DFT (razão 1,00031), e para $x$ real $\text{IDFT}(x) = \overline{\text{DFT}(x)}/N$. O cliente aplica o 1/1024 (`IFFT_HW_GAIN` em `morphe_config.py`), e o resultado coincide com o `idft.m` do Prof. Sanca a $2{,}5 \times 10^{-5}$.

## 6.5 Convolução, FIR e IIR

Os aceleradores em Q15.16 têm comportamentos numéricos próprios, descritos com o circuito no capítulo 3.

**Contra o que se compara.** Em todos os blocos a referência do aplicativo é a conta em ponto flutuante de 64 bits (`float64`, 53 bits de mantissa), feita sobre o mesmo sinal: `np.convolve` na convolução e no FIR, `np.fft.fft` na FFT e a mesma cascata de seções no IIR. O erro mostrado é, portanto, o custo do ponto fixo. Até 14/10/2026 o IIR era comparado com o modelo em ponto fixo, que faz a mesma conta do hardware e dava erro zero por construção; esse modelo continua rodando, só como alarme de defeito.

Em resumo:

**Convolução e FIR** somam os produtos inteiros em 64 bits e *arredondam* uma vez, na saída: o erro é de no máximo meio LSB, qualquer que seja o número de termos (desde 15/10/2026; antes truncavam cada produto, e o erro chegava a 521 LSB em 1024 por 1024). Não saturam: uma soma além de $\pm 32\,768$ dá a volta com o sinal trocado (seção 3.3.2).

**IIR** *arredonda* na saída de cada seção e *satura* em vez de dar a volta; o registrador `iir_error` avisa quando saturou. O resultado da placa coincide bit a bit com o modelo em ponto fixo em Python, e é comparado com a mesma cascata em ponto flutuante (seção 3.3.4).

**Os coeficientes dos filtros também são quantizados.** Um FIR projetado em ponto flutuante e convertido para Q15.16 muda cada coeficiente em no máximo meio passo ($7{,}6 \times 10^{-6}$, medido). No IIR, com os polos em seções de segunda ordem, o raio de cada polo se desloca no máximo cerca de $10^{-6}$, mesmo com polos em $|p| = 0{,}9984$. É isso que torna o Q15.16 suficiente para os dois.

## 6.6 O ADC

O conversor da placa tem 12 bits: 4096 níveis de 1 mV entre 0 e 4,095 V (ou entre −2,048 e +2,047 V no modo diferencial). O ruído de quantização de um conversor ideal de $B$ bits, com uma senoide que ocupa a faixa inteira, dá uma relação sinal-ruído de

$$\mathrm{SNR} = 6{,}02\,B + 1{,}76 \text{ dB} \approx 74 \text{ dB} \quad (B = 12).$$

O número de bits efetivos (ENOB) que a janela de aquisição mostra é essa fórmula invertida, aplicada à relação medida: $\mathrm{ENOB} = (\mathrm{SINAD} - 1{,}76)/6{,}02$. A medida usa a janela de Blackman-Harris, e não a de Hann: com a de Hann o vazamento espectral limitava a medida a cerca de 53 dB e escondia o conversor.

Uma senoide que ocupa só uma parte da faixa perde o mesmo que um sinal mal escalado na FFT: 6 dB a cada vez que a amplitude cai pela metade. Por isso a recomendação de usar amplitude perto de 2 V de pico em torno de 2 V de deslocamento (seção 3.4).

## 6.7 Números para referência rápida

Tabela 6.4: Erros medidos na placa, para comparação.

| Medida | Valor | Condição |
|---|---:|---|
| FFT → IFFT → sinal original, erro máximo | $6{,}1 \times 10^{-5}$ | as duas normalizadas |
| FFT → IFFT, erro máximo | $9{,}5 \times 10^{-4}$ | nenhuma normalizada |
| FFT contra o NumPy | 91,5 dB | normalizada |
| IFFT contra o NumPy | 92,7 dB | normalizada, espectro de pico 1280 |
| Convolução e FIR 1024 × 1024, erro máximo | 0,500 LSB | arredondamento, 15/10/2026 |
| Convolução e FIR contra o NumPy | 137 dB | amostras aleatórias em ±2 |
| Convolução 1024 × 1024, erro máximo | 521 LSB | truncamento, até 14/10/2026 |
| IIR contra o modelo em ponto fixo | 0 LSB | conferência do hardware |
| Coeficientes FIR, ponto flutuante contra Q15.16 | $\leq 7{,}6 \times 10^{-6}$ | meio passo |
| ADC, entrada no terra | 0,0000 V, desvio 0,03 mV | 24/09/2026 |
| ADC, entrada no 3,3 V da placa | 3,3300 V, desvio 2,6 mV | 24/09/2026 |
| ADC, senoide do gerador | ENOB 9,03 (56,1 dB) | 1 kHz, 07/10/2026 |

## 6.8 Regras práticas

1. Antes de codificar, olhe $\max|v|/32\,768$. Se for muito menor que 1, há bits sendo jogados fora.

2. Numa operação linear, normalize: a escala se desfaz exatamente.

3. Não espere mais que cerca de 92 dB de nada que passe pelo núcleo da FFT. Um erro de ida e volta de $10^{-5}$ não é defeito, é esse teto.

4. Um espectro não pode ser completado com zeros como um sinal no tempo.

5. Erro grande e sistemático é escala errada; erro pequeno e espalhado é quantização.

6. Na convolução, saída negativa para entradas positivas é estouro do acumulador, não defeito: reduza a amplitude.

# Capítulo 7

# O aplicativo cliente

O aplicativo é o que o aluno vê. Escrito em Python, com Tkinter para as janelas e Matplotlib para os gráficos, roda em qualquer PC do laboratório e não precisa de nenhum software da Intel. As dependências são o NumPy e o Matplotlib; o SciPy é opcional e só é usado pelo projeto de filtros IIR elípticos.

## 7.1 Abrir o aplicativo

Na instalação de turma (capítulo 9):

```
cd /opt/morphe/Python && python3 morphe_app.py
```

A janela principal abre já escolhendo a placa: entre as do `placas.conf` e as que a instalação conhece, fica com a livre e menos usada, e varre a rede em segundo plano atrás de placas fora da lista (capítulo 8).

O painel **Placas do laboratório**, no topo, mostra uma linha por placa, atualizada a cada 4 s: um ponto de cor (verde livre, âmbar ocupada, cinza sem resposta, vermelho com a FPGA não preparada), o nome, o IP e o que a placa está fazendo — "ocupada: convolução há 0,1 s ů 3 computadores no último minuto". Em cima, um resumo ("2 placas ů 1 livre ů 1 ocupada") e o botão *Procurar de novo*. A placa em uso é marcada com "você está aqui"; as outras têm um botão *Usar*, para trocar de placa com um clique. Sem placa nenhuma, o painel diz isso numa linha, sem janela de erro: o gerador de sinais, o comparador e os projetistas de filtro funcionam sem placa. O endereço e a porta, para conectar à mão, ficam recolhidos em *Conexão manual*.

## 7.2 As janelas

A janela principal é um menu de botões, um por ferramenta:

**Gerador de Sinais** monta um sinal e o mostra, sem placa. Serve para explorar e para salvar sinais em arquivo.

**Aquisição (ADC da placa)** lê tensões e captura sinais pela entrada analógica da placa (seção 7.3).

**Analisador espectral** o sinal do ADC passando pela FFT da FPGA, quadro a quadro, até o aluno clicar em *Parar* (seção 7.4).

**Áudio (codec)** o som entrando e saindo da placa pelo codec (seção 3.5). *Capturar* grava alguns segundos em blocos (256, 1024 ou 4096 quadros), mostra o último bloco e o espectro (em dB ou linear, com a janela escolhida, de 0 a $f_s/2$ ou de 0 a $f_s$ com a réplica) e salva em

`.wav` ou `.npy`. *Reproduzir* toca na LINE-out um sinal do gerador (senoide, quadrada, dente de serra, varredura, ruído branco), a última captura ou um arquivo; a opção "ajustar o volume" leva o pico a 90 % do fundo de escala. *Loopback* devolve o som pela saída da placa, ao computador ou aos dois. *Filtrar na placa* passa a última captura (ou um arquivo) por um filtro da FPGA, vindo do projetista IIR (no bloco IIR, em blocos de 1024 encadeados pelo estado do filtro) ou do projetista FIR (no bloco FIR, por *overlap-add*). Aberto desta janela, o projetista já vem com a $f_s$ do codec, 48 828,125 Hz; um filtro projetado para outra $f_s$ é recusado. O FIR pode ter mais de 1024 coeficientes, até 8191: o projetista aceita, e a placa aplica em blocos de 1024 coeficientes (cada bloco a mais é mais uma convolução por bloco do sinal). A janela mostra os espectros médios da entrada e da saída com $|H(f)|$, toca as duas e salva a saída; a caixa *Stem* desenha as amostras como hastes, sem ligar os pontos (na filtragem, 128 amostras do meio). *Salvar para comparar* grava entrada, filtro e saída da placa num bundle `.mrph`, que o Comparador abre e refaz no NumPy (FIR: convolução; IIR: a cascata em float64 e o modelo Q15.16 bit a bit), ou num `.npz` para um script próprio (`np.load`: `x`, `y_placa`, `fs` e `h` ou `sos`). Não é tempo real: o quadro *Medidas* mostra o atraso até ouvir (a gravação inteira, mais a filtragem, perto de 2 vezes o tempo real, mais o atraso de grupo do filtro). Filtrar em tempo real, com a saída na LINE-out, exige mudar a FPGA: é a ideia guardada na branch `estagio/filtro-tempo-real`. Uma seção recolhível resume o codec: $f_s$, o filtro *anti-aliasing*, os níveis e o atraso.

**Convolução** dois sinais, $x[n]$ e $h[n]$, convoluídos na FPGA. Mostra as entradas, a saída da placa, a referência do NumPy e o erro entre elas.

**FFT** um sinal, transformado na FPGA; mostra magnitude e fase, compara com o NumPy e permite voltar ao tempo pela IFFT da própria placa.

**IFFT** um espectro lido de arquivo (`.mrph`, `.npy` complexo ou texto com colunas real e imaginária), transformado de volta na FPGA. Serve para testar a inversa isolada do erro da direta.

**Filtro FIR** um sinal montado por superposição de componentes, filtrado na FPGA por um FIR que vem do projetista ou de um arquivo de coeficientes.

**Filtro IIR** a mesma ideia, com um IIR em seções de segunda ordem. O gráfico do meio é a resposta em frequência do projeto e do filtro quantizado. A saída da placa é comparada na hora com a mesma cascata em ponto flutuante de 64 bits, com os coeficientes de projeto: a barra de status mostra o erro máximo (também em LSB) e o SNR, que são o custo do ponto fixo. O modelo em ponto fixo, que tem de bater bit a bit com a placa, roda junto só como alarme: se divergir, a barra avisa em qual amostra.

**Comparar .mrph** abre um pacote salvo (seção 7.6) e refaz a conta no computador, sem placa.

Todas as janelas que usam a placa têm a mesma estrutura: um painel para o sinal de entrada, um botão que envia à FPGA, os gráficos (com barra de ferramentas para ampliar, deslocar e destacar o gráfico numa janela própria) e uma barra de status que diz o que aconteceu, inclusive o tempo que a placa levou.

Existe também uma janela de espectrograma, pronta e validada, que ficou fora do menu por decisão do supervisor; as linhas que a ligam estão comentadas no `morphe_app.py`, com a marca `ESPECTROGRAMA`.

### De onde vem o sinal

O painel de sinal de todas as janelas oferece sete tipos:

- cinco gerados: degrau, impulso, senoide, exponencial e retangular, com os seus parâmetros e o número de amostras $N$;
- **Arquivo**: um sinal qualquer, lido de `.csv`, `.txt`, `.npy` ou `.wav`. Texto com uma coluna (as amostras) ou duas (índice ou tempo, amostra), com vírgula, ponto e vírgula, tabulação ou espaços; aceita a vírgula decimal de planilhas em português. Do `.wav` vem também a taxa de amostragem. O leitor está em `Python/sinal_arquivo.py`;
- **Captura do ADC**: a última captura feita na janela de aquisição.

Com os dois últimos o tamanho do sinal é o do arquivo ou da captura, e pode passar muito de 1024 amostras: as janelas então processam por blocos (seção 7.7). Quando o sinal vem com uma taxa de amostragem e o filtro foi projetado com outra, as janelas do FIR e do IIR avisam. `exemplos/sinais/` tem arquivos de exemplo para cada janela, com um `README.md`.

### Os projetistas de filtros

**FIR.** O projetista (`fir_design.py`) tem dois modos. *Por especificação*, o padrão: o aluno dá o tipo (passa-baixa, passa-alta, passa-banda, rejeita-banda), a borda da banda passante, a largura da transição, o *ripple* e a atenuação; o programa escolhe a janela (retangular, Hann, Hamming, Blackman ou Kaiser) e calcula quantos coeficientes são necessários. É o método do `dsp_fir_filter.m` do Prof. Sanca, portado para Python. No modo *avançado*, o aluno escolhe o número de coeficientes e a janela. O teto é 1023 coeficientes (uma convolução da placa) quando aberto por outra janela, e 8191 quando aberto pelas janelas FIR e Áudio, que aplicam o filtro em blocos. Nos dois modos, o projetista mostra o que o filtro entrega *depois* da quantização em Q15.16. Validado na placa em 22/09/2026: 63,0 dB de rejeição medida contra 63,3 dB da referência.

**IIR.** O projetista (`iir_design.py`) segue o `dsp_iir_filter.m` do Prof. Sanca: Butterworth, Chebyshev I, Chebyshev II e elíptico, nos quatro tipos. A diferença deliberada é a saída: em vez de um polinômio de ordem $N$, ela é uma cascata de seções de segunda ordem, pelos motivos da seção 3.3.4. O Butterworth e os dois Chebyshev são calculados só com NumPy; o elíptico usa o SciPy e só aparece se ele estiver instalado. Quatro erros encontrados no `.m` original estão corrigidos no módulo Python e documentados nele.

Atenção à convenção do professor, mantida na janela: $f_s$ minúsculo (campos "fs rejeição", "fs1", "fs2") é a *borda da banda de rejeição*; a taxa de amostragem é o campo "Fs amostragem". Num passa-baixas a borda de rejeição tem de ficar acima de $f_p$; num passa-altas, abaixo. Se não ficar, a mensagem diz qual campo está fora de ordem e com que valores.

## 7.3 A janela de aquisição

A janela "Aquisição (ADC da placa)" depende de a placa ter o bitstream com o ADC; se não tiver, a própria mensagem de recusa do servidor aparece na barra de status (seção 4.3). Ela faz duas coisas.

**Voltímetro.** Lê a tensão média de 0,1 s na entrada escolhida, com desvio, mínimo e máximo, e pode repetir sozinho para acompanhar um ajuste. Não há hardware separado para isso: é uma captura de 2000 amostras a 20 kHz. O intervalo de 0,1 s contém um número inteiro de ciclos de 50 Hz e de 60 Hz, então a média cancela o zumbido da rede elétrica.

**Captura.** O aluno escolhe a entrada (um dos oito canais contra o terra, de 0 a 4,095 V, ou um dos quatro pares diferenciais, de ±2,048 V), a taxa de amostragem (de 1 a 200 kHz) e, no campo *Como*, de que jeito capturar:

- **número de amostras**: até 32 768, a captura é de uma vez, na memória da FPGA; acima disso, ou com 0 amostras (sem limite, até clicar em *Parar*), é contínua: a placa envia em blocos enquanto captura (seção 5.6);
- **gravar por um tempo**, em ms, s ou min: sempre contínua, com a tela mostrando o sinal enquanto ele chega; *Parar* antes do fim guarda o que já veio;
- **ao vivo**: contínua até *Parar*, com a tela atualizada dez vezes por segundo e as amostras descartadas, de modo que a memória não cresce. É o modo de só observar o sinal.

Nos dois últimos modos a tela mostra uma janela de tempo escolhida e pode sincronizar na subida do sinal, como um osciloscópio.

O gráfico mostra o sinal em volts; acima de 200 mil pontos, a envoltória. Para uma senoide, a janela mede frequência, SINAD e ENOB, que é o jeito de conferir o conversor com um gerador de funções (seção 3.4). A captura pode ser salva em `.csv`, `.npy` ou `.wav` e passa a estar disponível como o tipo "Captura do ADC" em todas as outras janelas. As contas — a palavra de configuração, a taxa real, a conversão para volts, as métricas e a gravação — estão em `Python/aquisicao.py`, separadas da janela.

A taxa real é 50 MHz/divisor, com divisor inteiro. Ela é exata para as taxas que dividem 50 MHz; as outras são arredondadas, e a janela mostra a taxa real usada (44,1 kHz, por exemplo, vira 44 091,7 Hz).

## 7.4 O analisador espectral

A janela "Analisador espectral" junta o ADC e a FFT da FPGA. Enquanto o aluno não clica em *Parar*, ela repete um ciclo, o *quadro*:

1. o ADC captura $N$ amostras à taxa escolhida, com $N$ igual a 1024, 4096 ou 16 384;
2. no computador, tira-se o nível DC (opcional, ligado por padrão) e aplica-se a janela;
3. a FPGA calcula a FFT: uma FFT de 1024 pontos ou, para $N$ maior, a FFT longa pelo algoritmo de quatro passos (seção 7.7), com $N/1024$ FFTs na placa;
4. a tela mostra o quadro no tempo e o espectro, com o pico marcado.

Captura e FFT são pedidos separados, como os de qualquer janela: entre um quadro e o seguinte, a placa atende quem mais estiver na fila.

**O espectro.** O gráfico mostra o **espectro completo**: os $N$ valores de $X[k]$ que a FPGA devolveu, de 0 a $f_s$, em **escala linear**, com o eixo vertical em volts de pico. Como o sinal é real, um tom de frequência $f$ aparece duas vezes, em $f$ e no espelho, em $f_s - f$; uma linha pontilhada marca $f_s/2$, onde o espelho começa. A amplitude é dividida pelo ganho da janela, $|X[k]| \,/\, (\sum w[n]/2)$, de modo que uma senoide de amplitude $A$ aparece com altura $A$ nas duas linhas, e o nível DC, que só aparece em $k = 0$, sai com o seu valor em volts. A resolução é $f_s/N$. O pico é procurado na metade de baixo, e a frequência dele é refinada por uma parábola nos três pontos em volta, melhor que o ponto inteiro, que erra até meia resolução. A escala vertical se ajusta a cada quadro ao maior valor do espectro.

**Janela e média.** Há quatro janelas: Hann (a padrão), Blackman-Harris, *flat-top* e retangular. A diferença entre elas aparece num tom que cai entre dois pontos da FFT: a amplitude dele sai até 15 % abaixo com a Hann, 9 % com a Blackman-Harris e 36 % com a retangular. A *flat-top* erra menos de 0,3 % em qualquer frequência, ao custo de um pico mais largo: é a janela de medir amplitude. Mudar a janela mostra isso na hora, e é um bom exercício. A média pode ser de 4 ou 16 quadros (média de potência, que reduz a flutuação do ruído) ou a retenção de pico, que guarda o maior valor de cada ponto desde o início.

**A conferência.** A cada quadro, o NumPy calcula a FFT do mesmo $x[n]$ janelado, e o painel *Medidas do quadro* mostra quanto as duas concordam: é a prova, quadro a quadro, de que o espectro saiu da FPGA. O mesmo painel dá a frequência e a amplitude do pico, o nível DC, os tempos de captura e de FFT, quantos quadros por segundo e se a entrada saturou. Em 30/09/2026, na placa 1, com quadros de 1024 pontos: FPGA e NumPy concordaram com 97,6 dB de relação sinal-erro, a 5,6 quadros por segundo.

**Salvar.** *Salvar quadro* grava o último quadro num `.mrph` (o $x[n]$ como foi enviado à FPGA e o $X[k]$ que voltou), que o comparador abre (seção 7.6).

Como a placa não tem filtro *anti-aliasing*, um tom acima de $f_s/2$ na entrada aparece no espectro numa frequência falsa; a janela lembra isso ao lado da taxa escolhida.

## 7.5 Os módulos

As janelas são finas; o que importa está nos módulos que elas usam.

Tabela 7.1: Os módulos do cliente, em `Python/`.

| Módulo | O que faz |
|---|---|
| `morphe_app.py` | a janela principal e o menu |
| `*_window.py` | uma janela por ferramenta |
| `signal_panel.py` | o painel de sinal comum a todas as janelas |
| `tcp_panel.py` | o painel "Placas do laboratório": descoberta, escolha e estado das placas |
| `morphe_protocol.py` | o protocolo (capítulo 5): monta e decodifica as mensagens, abre as conexões, acha as placas na rede |
| `morphe_config.py` | os limites do hardware, espelho do `C/morphe_config.h` |
| `dsp_core.py` | geração de sinais, conversão para ponto fixo, a referência em ponto flutuante e a leitura e escrita de `.mrph` |
| `blocos.py` | o processamento de sinais maiores que o hardware (seção 7.7) |
| `sinal_arquivo.py` | a leitura de sinais de arquivo |
| `aquisicao.py` | as contas do ADC |
| `audio.py` | o áudio do codec: captura em blocos, reprodução, *loopback* |
| `fir_design.py`, `iir_design.py` | os projetistas, o modelo em ponto fixo do IIR que reproduz a placa bit a bit e a mesma cascata em `float64`, a referência da comparação |
| `morphe_theme.py`, `plot_toolbar.py`, `popout_helper.py` | aparência, barra dos gráficos, gráficos em janela própria |
| `morphe_export.py` | exporta uma janela aberta para SVG vetorial, para figuras de artigos e deste manual |

## 7.6 O pacote `.mrph` e o comparador

O `.mrph` é o formato de arquivo do Morphe: texto puro, legível em qualquer editor. Um arquivo simples tem um cabeçalho com linhas iniciadas por `#` (data, tipo numérico, tamanho, taxa de amostragem, descrição) e uma tabela de valores, com duas colunas para sinais reais ($n$, valor) e três para complexos ($k$, real, imaginário). Um *pacote* reúne vários sinais num arquivo só, em seções iniciadas por `## section:`, cada uma com o seu cabeçalho: numa convolução, $x$, $h$ e $y$; numa FFT, $x$ e $X$.

Os pacotes vêm de dois lugares: as janelas salvam o que o aluno fez, e o servidor grava um a cada operação na placa (seção 4.3 e seguinte). O comparador abre qualquer um deles, identifica a operação, refaz a conta em NumPy (no IIR, no modelo em ponto fixo) sobre a mesma entrada e mostra a diferença. É o caminho para estudar um resultado depois, sem placa, ou para investigar um resultado estranho que um aluno trouxe. `exemplos/` tem pacotes reais produzidos pela placa.

O comparador abre FFTs de qualquer tamanho, não só de 1024 pontos: a referência é calculada com o tamanho do próprio arquivo. Assim ele confere também os quadros de 4096 e 16 384 pontos do analisador espectral e as FFTs longas das outras janelas.

## 7.7 Sinais maiores que o hardware: processamento em blocos

Os aceleradores processam no máximo 1024 amostras por vez. Um arquivo de áudio de um segundo tem 44 100; uma captura do ADC pode ter milhões. O `Python/blocos.py` divide o sinal em blocos, manda cada um à placa pelo mesmo caminho que as janelas usam para um bloco só, e junta os resultados. São três técnicas, uma por tipo de operação, e cada uma tem um argumento de por que o resultado é o mesmo que uma operação única daria.

Todas as funções de bloco recebem a operação de *um* bloco como parâmetro: na placa, as classes `ConvPlaca`, `FirPlaca`, `FftPlaca`, `IfftPlaca` e `IirPlaca`; nos testes sem placa, o `np.convolve`, o `np.fft.fft` e o modelo em ponto fixo. Isso permite conferir a montagem dos blocos separada do hardware.

### 7.7.1 Convolução e FIR: *overlap-add*

A convolução é linear e invariante no tempo. Cortando $x$ em blocos $x_i$ de $L$ amostras,

$$x[n] = \sum_i x_i[n - iL] \quad \Longrightarrow \quad (x * h)[n] = \sum_i (x_i * h)[n - iL].$$

Cada $x_i * h$ é uma convolução que cabe na placa, com $L + M - 1$ amostras de saída; as caudas de $M - 1$ amostras de blocos vizinhos se sobrepõem e são somadas (Figura 7.1). Se $h$ também passa de 1024 amostras, é cortado do mesmo jeito, e cada par de pedaços vira uma requisição. O resultado é exatamente a convolução linear: a única diferença para uma operação única é a quantização em Q15.16, que já existia.

### 7.7.2 FFT e IFFT: o algoritmo de quatro passos

Uma DFT de $N = M \cdot 1024$ pontos se decompõe em DFTs de 1024 e DFTs de $M$ pontos, ligadas por fatores de giro. É a decomposição de Cooley–Tukey com $N_1 = M$ e $N_2 = 1024$. Com $n = n_1 + Mn_2$ e $k = 1024\,k_1 + k_2$:

$$X[1024\,k_1 + k_2] = \sum_{n_1=0}^{M-1} W_M^{n_1 k_1}\; W_N^{n_1 k_2}\; \underbrace{\mathrm{FFT}_{1024}\{x[n_1 + Mn_2]\}[k_2]}_{\text{na placa}}.$$

Em quatro passos (Figura 7.2):

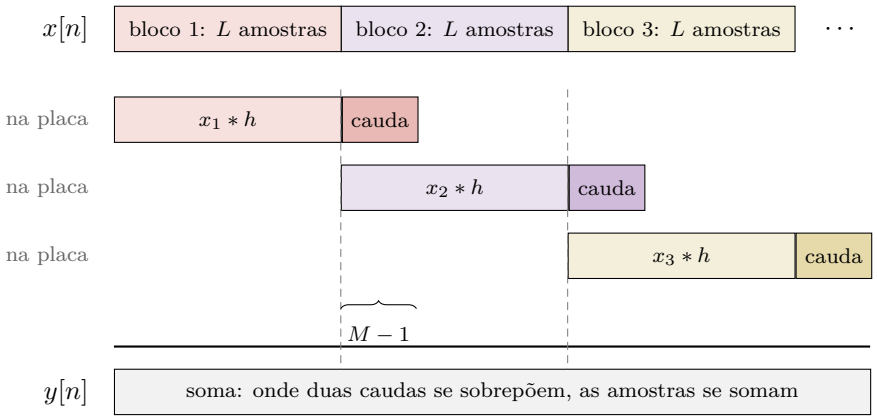



Figura 7.1: *Overlap-add*: cada bloco de $L$ amostras é convoluído na placa; as caudas de $M-1$ amostras se somam.

1. a placa faz $M$ FFTs de 1024 pontos, uma por subsequência $x[n_1], x[n_1+M], x[n_1+2M], \ldots$;
2. cada resultado é multiplicado pelos fatores de giro $W_N^{n_1 k_2}$;
3. o PC faz 1024 DFTs de $M$ pontos, uma por coluna $k_2$;
4. a matriz resultante, lida por linhas, é $X[k]$ na ordem natural.

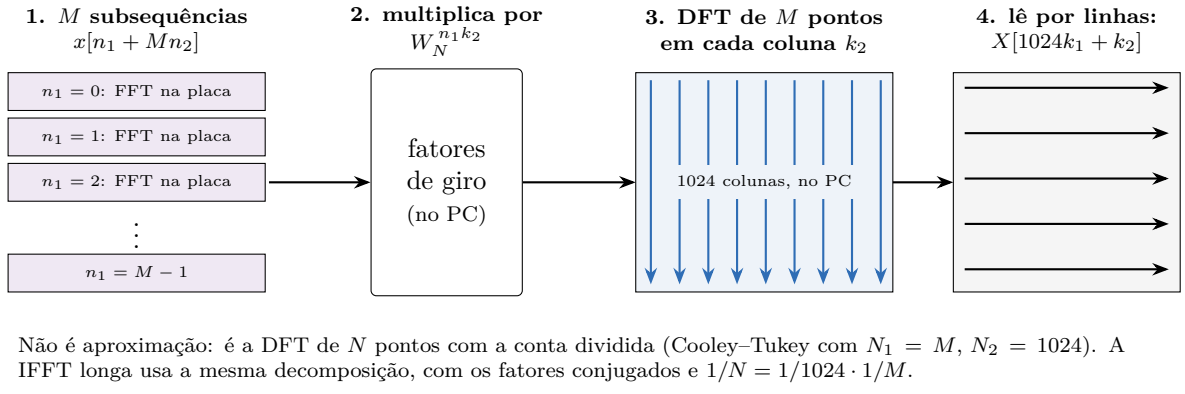



Figura 7.2: FFT de $N = M \cdot 1024$ pontos pelo algoritmo de quatro passos: $M$ FFTs de 1024 na placa, fatores de giro e DFTs de $M$ pontos no PC.

Não é uma aproximação: é a mesma DFT, com a conta dividida. A precisão é a das FFTs da placa, cada uma normalizada por conta própria, mais a do ponto flutuante de 64 bits do PC, desprezível perto dela. O sinal é completado com zeros até o múltiplo de 1024 acima. A IFFT longa é a mesma decomposição no sentido inverso, e por isso exige que o espectro já tenha um número de pontos múltiplo de 1024.

A escolha foi deliberada: um espectrograma (FFTs de quadros sobrepostos) também processaria sinais longos, mas mudaria o significado da janela da FFT, que é mostrar *a* transformada do sinal.

### 7.7.3 IIR: blocos encadeados pelo estado

O IIR não se decompõe assim. A saída em $n$ depende de toda a entrada anterior, e essa memória inteira cabe no *estado* do filtro: quatro números por seção. Desde 19/10/2026 o bloco da placa devolve o estado final de cada execução e aceita o inicial (seção 5.5); o cliente passa o final de um

bloco como inicial do seguinte. Os blocos têm as 1024 amostras inteiras, não há aquecimento, e a saída é a do filtro rodando sem parar, bit a bit — qualquer que seja a posição dos polos. O modelo em ponto fixo faz o mesmo, e o alarme bit a bit da janela compara a placa com o filtro contínuo.

### 7.7.4 IIR: blocos com aquecimento (bitstream sem estado)

Com um bitstream anterior, que zera o estado a cada execução, o cliente percebe na primeira resposta e volta ao esquema antigo: o estado é *reconstruído*. Cada bloco começa $A$ amostras *antes* do trecho que interessa, e as primeiras $A$ saídas são descartadas (Figura 7.3). Se o aquecimento for longo o bastante para o transitório do estado zero morrer, o trecho guardado coincide com o do filtro rodando sem parar.

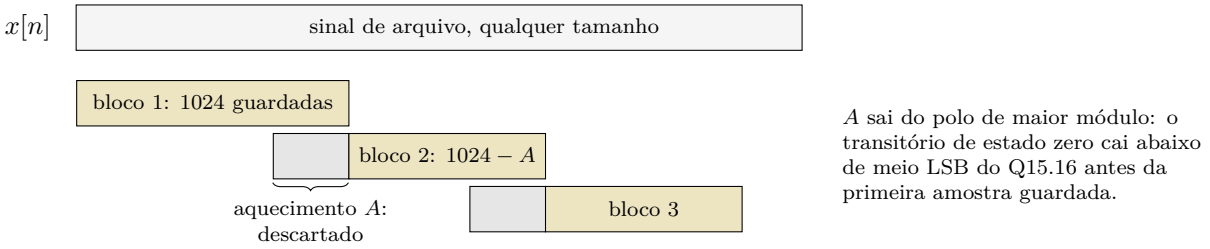



Figura 7.3: IIR por blocos: cada bloco começa algumas amostras antes (*aquecimento*) e essa parte da saída é descartada.

O aquecimento é calculado pelo polo de maior módulo $|p|$: o transitório decai como $|p|^n$, e o critério é ele cair abaixo de meio passo do Q15.16,

$$A = \left\lceil 1{,}5 \cdot \frac{\ln 2^{-17}}{\ln |p|} \right\rceil + 8 \cdot (\text{número de seções}),$$

com a folga de 50 % e o termo por seção cobrindo os polos repetidos. Um filtro com polos muito perto do círculo unitário pode precisar de um aquecimento maior que o próprio bloco; nesse caso o cliente recusa com uma mensagem que explica por quê e pede o bitstream novo. Era a limitação que o estado removeu: a $f_s = 48{,}8$ kHz do codec, um Butterworth de ordem 4 com corte em 200 Hz pede 1811 amostras de aquecimento, e pelo estado roda sem nenhuma.

Para o alarme bit a bit da janela continuar valendo, o modelo em ponto fixo roda com *os mesmos blocos*: placa e modelo batem exatamente. A comparação com o `float64` usa o filtro contínuo, sem blocos. A distância entre os blocos e o filtro contínuo é medida à parte e mostrada em LSB. Em ponto fixo ela nem sempre chega a zero, porque o arredondamento na realimentação pode fazer estados iniciais diferentes convergirem para bits diferentes: medido em 23/09/2026, 0 LSB num Butterworth de ordem 4 com corte em 1 kHz e 4 a 8 LSB num de ordem 2 com corte em 100 Hz, ambos a 8 kHz.

### 7.7.5 O custo

Cada bloco é uma requisição, e a placa atende cerca de 8 por segundo (capítulo 8). Uma convolução de 5000 amostras por um filtro de 67 coeficientes são 5 requisições; um FIR sobre 5 s de áudio a 44,1 kHz são 216, perto de meio minuto. Enquanto isso, a barra de status das janelas mostra o bloco em curso (“Bloco 3 de 5 na FPGA”).

# Capítulo 8

# Várias placas, vários alunos

O laboratório tem duas placas e uma turma. Este capítulo descreve o que acontece entre um cliente e as placas quando várias pessoas usam o sistema ao mesmo tempo: como o cliente acha as placas, como escolhe uma, o que a fila do servidor faz com o tempo de resposta e o que foi medido. O texto completo, com as mensagens de diagnóstico, está em `docs/GERENCIAMENTO-PLACAS.md`.

## 8.1 Descoberta

O cliente junta as placas que conhece de três fontes, sem repetir:

`placas.conf` na raiz do repositório, **versionado**: a lista das placas do laboratório, um IP por linha, com um comentário (placa, *hostname*, MAC; até 22/10/2026 também o cabo JTAG, que saiu porque o nome dele vem da porta USB da estação e muda com ela). Ao lado dele, o `placas.local.conf`, fora do git, recebe as placas que um laboratório acrescenta sem acesso de escrita ao repositório; os dois são lidos juntos, e o `atualizar-lab.sh` copia o da estação para os computadores. É igual em todo computador, e é o que faz um computador recém-instalado achar as placas só com o `git clone`. O *hostname* do comentário é o que identifica cada placa. O IP escrito ali é só o ponto de partida: se o DHCP o trocar, a busca por *broadcast* (abaixo) acha a placa no endereço novo. Corrigir o IP no arquivo continua útil, porque deixa a conexão imediata, mas desde 30/09/2026 não é mais obrigatório.

`.morphe-estado/placas` local: toda placa que o `morphe-up.sh` já preparou naquela instalação; a última fica também em `.morphe-estado/placa`.

**a busca na rede** primeiro, **um único pedido por *broadcast*** à porta de estado (seção 8.2), enviado a toda a rede do laboratório (`172.16.255.255`) e a `255.255.255.255`. Cada placa responde com o seu IP e o seu *hostname*, em menos de um segundo, qualquer que seja o endereço que o DHCP lhe deu. Testado na estação em 30/09/2026: as duas placas responderam. Só se nenhuma responder — rede que bloqueia *broadcast*, ou servidor anterior a 28/09/2026 — vale a varredura antiga: um `PING` a cada endereço das sub-redes prováveis (a da placa conhecida, a do próprio computador e as usadas historicamente no laboratório), com até 64 conexões em paralelo e prazos curtos, que leva alguns segundos.

As listas e a busca trabalham juntas:

- **ao abrir**, o aplicativo conecta pela lista, na hora, e procura na rede em segundo plano. Uma placa preparada que apareça fora da lista — quase sempre uma que trocou de IP por DHCP — entra na lista local e passa a disputar a escolha a partir da próxima abertura;
- **se nenhuma placa da lista responder**, a busca decide: o aplicativo escolhe entre todas as que ela achar;

- **o botão *Procurar de novo*** junta a lista e uma busca completa, e escolhe entre todas.

O `morphe-up.sh` usa o mesmo *broadcast*, mas procura pelo nome: o `--todas` passa por cada placa da lista e a acha pelo *hostname*, no IP que ela tiver naquele dia (capítulo 9).

Não se deve usar o `morphe-up.sh` só para registrar uma placa: mesmo sem reprogramar a FPGA, ele reinicia o servidor e interrompe quem estiver no meio de uma operação.

## 8.2 A porta de estado

O servidor atende uma conexão por vez, então uma placa ocupada não responde nem ao `PING` até terminar o que está fazendo. Por muito tempo o cliente não tinha como distinguir "placa ocupada" de "placa fora do ar": nos dois casos a sonda simplesmente demorava.

Desde 28/09/2026 o servidor tem uma *thread* que responde por UDP, na porta 5001 (a porta TCP mais um), **na hora**, mesmo com a placa ocupada. Um datagrama que comece com `MRPS` recebe um texto com: livre ou ocupada; qual operação e há quanto tempo; o endereço do computador que a pediu; quantos computadores usaram a placa no último minuto (o `PING` não conta, porque todo cliente sonda todas as placas ao abrir); quantas operações ela já atendeu; e se a FPGA está preparada. A *thread* não toca a FPGA: lê só variáveis do próprio servidor e a marca de FPGA preparada, então não há como ela travar o barramento.

Medido em 28/09/2026, nas duas placas, da estação (sub-rede `172.16.101.x`) para as placas (`172.16.230.x`): a resposta chega em 1,1 ms com a placa livre e em 1,4 ms durante uma captura contínua do ADC, enquanto um `PING` enviado no mesmo instante fica na fila sem resposta. O teste é o `Python/ferramentas/testa_porta_estado.py`.

Três coisas usam a porta de estado:

- a escolha de placa, na seção seguinte;
- o painel *Placas do laboratório* do aplicativo, que a cada 4 s mostra uma linha por placa, com o que cada uma está fazendo ("ocupada: convolução há 0,1 s ů 2 computadores no último minuto"), e deixa o aluno trocar de placa com o botão *Usar* (capítulo 7);
- o `Python/ferramentas/estado_placas.py`, para o professor ou o monitor verem todas as placas de uma vez; com `--seguir`, a tela se atualiza a cada segundo.

## 8.3 Escolha da placa

Com mais de uma placa conhecida, o cliente consulta a porta de estado de todas ao mesmo tempo e fica com a primeira pela ordem: **livre antes de ocupada**, depois **a que teve menos computadores no último minuto**, depois a que respondeu mais rápido. Placas com a FPGA não preparada nunca são escolhidas. O critério dos computadores recentes é o que divide a turma entre as placas: sem ele, duas placas livres empatariam e todos cairiam na que respondesse primeiro. O painel diz, ao conectar, o que as outras placas estavam fazendo.

Se nenhuma placa responder na porta de estado — servidor anterior a 28/09/2026, ou UDP bloqueado na rede —, vale o método antigo (Figura 8.1): um `PING` a todas ao mesmo tempo, e fica a primeira que responder. O tempo do `PING` também mede ocupação, pela mesma razão de antes: uma placa ocupada só responde depois de terminar.

A escolha acontece quando o aplicativo abre (ou em *Procurar de novo*). Ela não é uma reserva: dois alunos que abram o aplicativo no mesmo instante ainda podem cair na mesma placa, e uma placa escolhida livre pode ficar ocupada depois.

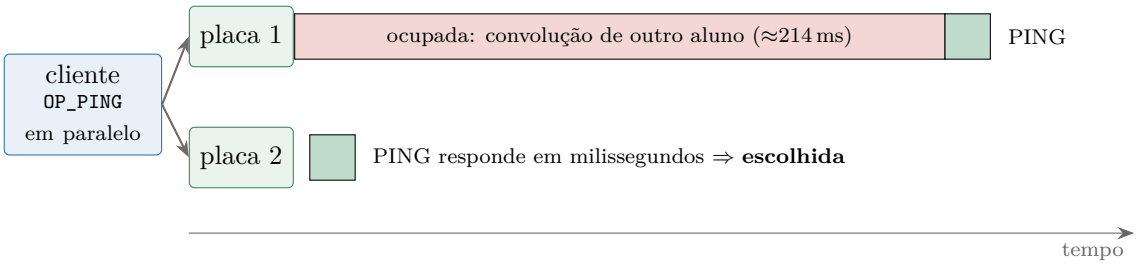



Figura 8.1: O método antigo de escolha, que vale quando nenhuma placa responde na porta de estado: o cliente sonda todas em paralelo e fica com a que responder primeiro. Uma placa ocupada só responde ao fim da operação em curso.

## 8.4 A fila e o tempo de resposta

Como o servidor atende uma conexão por vez, dois alunos na mesma placa **não recebem erro: esperam**. A segunda requisição fica na fila do sistema operacional até a primeira terminar, e os tempos se somam. Com $k$ clientes disputando a mesma placa, cada um vê cerca de $k$ vezes o tempo de placa livre.

A fila do `listen(4)` guarda cinco conexões, e a sexta é a que está sendo atendida. O que passa disso não é recusado: no Linux, o sistema descarta o pedido de conexão em silêncio e o cliente o repete cerca de um segundo depois. O sintoma do excesso é, portanto, uma cauda de latência, não um erro.

## 8.5 O que foi medido

As medições de 22/09/2026 usaram o `Python/ferramentas/testa_concorrencia.py`, que dispara requisições simultâneas e **confere cada resposta**: uma convolução com impulso tem de devolver a própria entrada, e a FFT de um impulso tem de dar espectro plano. Não bastava cronometrar; era preciso saber se alguma resposta chegava trocada.

Tabela 8.1: Vários clientes simultâneos, operações de 1024 amostras, metade convoluções (≈215 ms) e metade FFTs (≈21 ms). Placa livre: mediana de 120 ms (22/09/2026).

| Cenário | Respostas certas | Mediana | p95 | Pior |
|---|---|---|---|---|
| 4 clientes, uma placa | 32/32 | 471 ms (3,9×) | — | — |
| 6 clientes, uma placa | 24/24 | 604 ms (5,1×) | — | 1,29 s |
| 8 clientes, uma placa | 64/64 | 517 ms | 1,29 s | 6,39 s |
| 10 clientes, uma placa | 80/80 | 519 ms | 4,56 s | 7,07 s |
| 3 clientes em cada uma de duas máquinas, duas placas | 90/90 | — | — | — |

Três conclusões:

- **Nenhuma resposta foi perdida nem trocada**, em nenhum cenário. As 90 requisições do teste com duas máquinas e duas placas foram conferidas uma a uma.
- **Uma placa atende cerca de 8 requisições por segundo** com essa mistura de operações, e esse número não muda de 2 para 10 clientes: é o tempo médio de uma operação (120 ms) invertido. A partir de 8 clientes a placa está saturada, e cliente a mais só acrescenta espera. Duas placas dão cerca de 16 requisições por segundo para a turma inteira.

- O excesso aparece como cauda: de 8 para 10 clientes a mediana não se move e o p95 triplica.

Dois clientes gráficos, um em cada máquina, escolheram placas diferentes.

**A captura do ADC ocupa a placa.** Uma captura contínua mantém a conexão aberta enquanto dura, e durante esse tempo a placa não atende mais ninguém: outro aluno na mesma placa espera. A escolha automática não a escolhe (a porta de estado diz "ocupada: captura contínua do ADC"), e o painel de quem já estava nela mostra por que tudo parou. Numa aula com o ADC, é melhor reservar uma placa para a aquisição.

Em 28/09/2026 a mesma medida de 6 clientes numa placa foi repetida com o servidor que tem a porta de estado: 48/48 respostas conferidas, 8,4 requisições por segundo. A *thread* a mais não mudou nada no atendimento.

## 8.6 O que não existe

Estas são as limitações conhecidas da infraestrutura. Nenhuma impede o uso em turma com supervisão; todas impedem o uso sem supervisão com mais alunos do que placas.

1. **Reserva.** A escolha pela porta de estado divide a turma entre as placas, mas não reserva nenhuma: duas pessoas ainda podem usar a mesma placa, e uma placa escolhida livre pode ficar ocupada depois. O resultado sai sempre certo, porque cada requisição é completa; o que muda é o tempo, e agora o painel diz por quê. Uma reserva com prazo, renovada pelo cliente, exigiria estado no servidor e faria alunos esperarem por placa parada — com duas placas e uma turma, foi considerada pior que o que existe.

2. **Troca automática de placa.** A escolha acontece ao abrir o aplicativo. Se a placa escolhida lotar, o painel mostra isso, mas quem troca é o aluno, com o botão *Usar* de outra placa.

O **endereço fixo** estava nesta lista até 30/09/2026: as placas pegam IP por DHCP, e cada troca exigia corrigir o `placas.conf`. Com a busca por *broadcast*, o IP deixou de importar, e a reserva de DHCP na TI, antes proposta, ficou desnecessária.

## 8.7 Sintoma, causa e o que fazer

Tabela 8.2: Sintomas ligados a várias placas e vários clientes.

| **Sintoma** | **Causa** | **O que fazer** |
|---|---|---|
| Tudo ficou duas vezes mais lento | outra pessoa está na mesma placa; o painel mostra "ocupada" | é a fila; *Usar* numa placa livre |
| "A placa não respondeu", mas ela está no ar | placa ocupada (uma captura contínua, por exemplo) | `estado_placas.py` diz o quê e de quem |
| `estado_placas.py`: "responde ao PING, mas não na porta de estado" | servidor antigo, ou UDP bloqueado na rede | `./morphe-up.sh --deploy` |
| O `morphe-up.sh` avisa que a placa não está em `placas.conf` | o IP dela mudou por DHCP | nada urgente: a busca a acha; corrigir o IP no `placas.conf` deixa a conexão imediata |
| O aplicativo leva alguns segundos para achar as placas | ninguém respondeu ao *broadcast* e valeu a varredura antiga | conferir com `estado_placas.py`; servidor antigo pede `./morphe-up.sh --deploy` |
| Operação recusada com "FPGA não preparada" | a placa foi ligada e ninguém programou o bitstream | `./morphe-up.sh` na estação |
| Um computador só enxerga uma placa | o aplicativo dele está desatualizado | `/opt/morphe/atualizar.sh` |
| "o *tether* de DE-SoC [1-x] pertence à conta . . . " | outra conta preparou aquela placa | usá-la como está ou pedir àquela conta um `--down` |

# Capítulo 9

# Preparação e operação do laboratório

Este capítulo é para quem prepara a aula: o professor, o monitor, ou o aluno que chegou primeiro. Descreve a rotina diária, o que o `morphe-up.sh` faz, a instalação para a turma, o Quartus nos computadores e como pôr uma placa nova em serviço. Os detalhes, com o histórico de cada decisão, estão em `PREPARACAO.md` e `docs/NOVA-PLACA.md`.

## 9.1 A rotina

Com as placas ligadas à rede e aos cabos USB-Blaster da estação, e a instalação de turma feita (seção 9.4), a aula começa com um comando na estação, que prepara todas as placas:

```
cd /opt/morphe && ./morphe-up.sh --todas
```

e cada aluno, em qualquer computador, abre o aplicativo:

```
cd /opt/morphe/Python && python3 morphe_app.py
```

O `--todas` grava, pelo USB, a FPGA de toda placa ligada à estação (o mesmo `programar_local.py` do aplicativo, que não depende do nome do cabo) e depois prepara, uma a uma, cada placa do `placas.conf` e do `placas.local.conf`, achada pelo *hostname*, só pela rede (`--skip-fpga`). No fim diz quais ficaram prontas; uma placa que falhe não impede as outras. Em 30/09/2026, da conta `alunopds`: 2 de 2 prontas (ainda no esquema antigo, uma rodada por cabo). O IP não precisa ser informado: o script acha a placa pelo nome. Desde 22/10/2026 o próprio aplicativo também grava, ao abrir, as FPGAs ligadas pelo USB no computador em que ele roda (`docs/GERENCIAMENTO-PLACAS.md`, item 6). Com uma placa só, o `./morphe-up.sh` sem argumentos basta. Depois de preparada, a placa fica pronta até ser desligada ou até a estação ser desligada (seção 9.3).

Antes dessa rotina, preparar a plataforma levava 9 passos, dois deles com SSH na placa e cinco pontos de conhecimento que só o autor tinha; a contagem está em `docs/LINHA-DE-BASE-PASSOS.md`. Hoje são 3, e um deles é a própria operação.

## 9.2 O que o `morphe-up.sh` faz

A Figura 9.1 mostra os passos, na ordem em que o script os executa e imprime. Ele para no primeiro problema, com uma mensagem que diz o que fazer. Alguns passos merecem explicação.

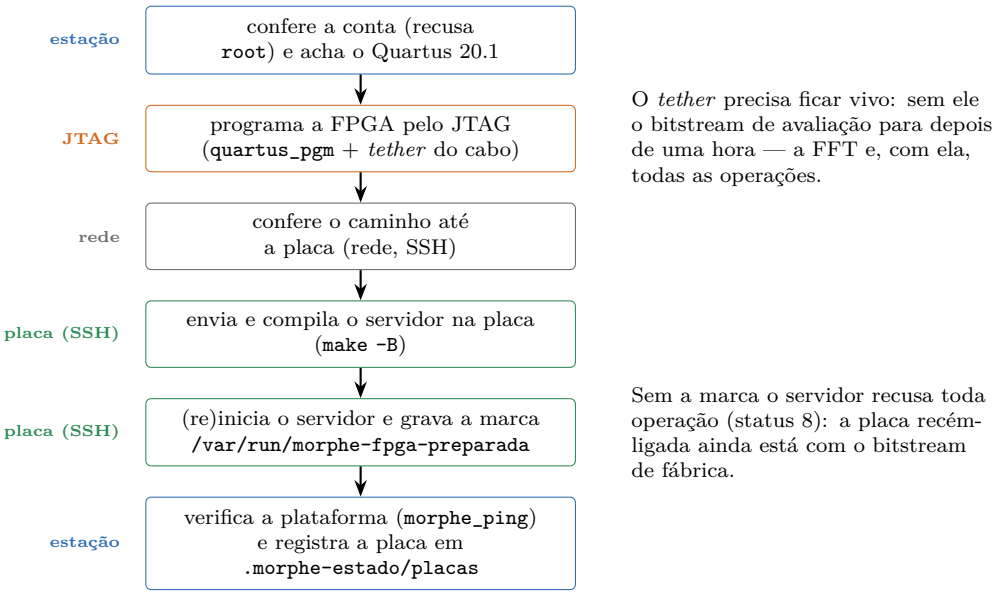



Figura 9.1: O que o `morphe-up.sh` faz, em ordem.

**Achar o Quartus.** O script procura no `PATH`, em `$QUARTUS_ROOTDIR` e nas instalações usuais, e prefere a versão **20.1**, a que compilou o bitstream. Outra versão só é usada, com aviso, se não houver a 20.1 em lugar nenhum. Ele também **recusa rodar como `root`**: em 23/09/2026 um `sudo ./morphe-up.sh` achou o Quartus 22.1 da conta `root`, deixou o *tether* como processo do `root`, que nenhuma outra conta consegue encerrar, e pediu a senha da placa três vezes.

**Conferir se a placa está em uso.** Antes de mexer em qualquer coisa, o script pergunta à porta de estado da placa (seção 8.2) se ela está ocupada e quantos computadores a usaram no último minuto. Havendo uso, mostra o que é ("agora: convolução há 0,3 s, pedida por 172.16.101.40") e pergunta se deve continuar, porque reprogramar a FPGA, ou só reiniciar o servidor, interrompe quem estiver no meio de uma operação. Sem terminal para perguntar, ele para. A placa conferida é a do `--board`; com só o `--cable`, a que o `placas.conf` associa àquele cabo; senão, a lembrada. Placa que não responde na porta de estado (desligada, ou com servidor antigo) não é conferida. O `--forca` pula a pergunta.

**Achar a placa do cabo.** A FPGA é programada pelo cabo JTAG, mas o servidor é reiniciado pela rede, e as duas coisas têm de ser a mesma placa. Desde 22/10/2026 quem garante isso é a própria placa: depois de subir o servidor, o script pergunta a ela se a FPGA *dela* tem a assinatura do bitstream (`fpga_assinatura` no `PING`), e para se a resposta for não — o cabo gravou outra placa. Antes, a garantia era a coluna do cabo no `placas.conf`, que ficava errada sempre que um cabo trocava de porta USB, e o script gravava a marca de "preparada" na placa errada.

**Descobrir o cabo.** O nome do cabo JTAG (`DE-SoC [1-1]`, `[1-2]`, ...) depende da porta USB. O script o lê do `jtagconfig`, em vez de supor.

**Programar e manter o *tether*.** Ver a seção 9.3.

**Servidor sempre depois da FPGA.** O servidor é (re)iniciado depois de a FPGA ser programada, nunca antes, e o script grava a marca `/var/run/morphe-fpga-preparada` na placa (seção 4.3). O servidor é reenviado e recompilado se algum dos seus seis arquivos mudou.

**Verificar.** O `Quartus/morphe_ping.py` faz quatro testes contra a placa: conexão, `PING`, uma convolução de impulso (que tem de devolver a própria entrada) e uma FFT de impulso (que tem de dar espectro plano). O script só devolve o controle com os quatro passando, e traduz as falhas: falhar os dois primeiros é rede; passar os dois primeiros e falhar os dois últimos é FPGA ou `hps_0.h`.

**Registrar.** A placa vai para `.morphe-estado/placas`, que é o que os clientes leem (capítulo 8).

Tabela 9.1: As opções do `morphe-up.sh`.

| Opção | O que faz |
|---|---|
| (nenhuma) | prepara a placa lembrada, ou a que achar na rede |
| `--todas` | grava pelo USB as FPGAs ligadas à estação e prepara, uma a uma, as placas do `placas.conf` e do `placas.local.conf`, pelo nome |
| `--cable` *nome* | usa este cabo JTAG, quando há mais de um |
| `--marca` | para um `.sof` sem a assinatura (anterior a 22/10/2026): grava a marca de "preparada", como antes |
| `--board` *ip* | prepara a placa neste IP (normalmente desnecessário), e passa a lembrá-la |
| `--deploy` | reenvia e recompila o servidor mesmo sem mudança |
| `--skip-fpga` | só o servidor, sem reprogramar a FPGA |
| `--forca` | não pergunta, mesmo com a placa em uso |
| `--codec` *entrada* | o codec de áudio com a entrada `mic` (MIC-in, o padrão), `linha` (LINE-in) ou `nao` (não mexe no codec) |
| `--codec-volume` *dB* | volume da LINE-out, de −73 a +6 (padrão −20) |
| `--setup-ssh` | uma vez por placa e por conta: instala a chave SSH (seção 9.5) |
| `--status` | mostra, placa a placa, o que está no ar e se o *tether* está vivo, e as placas que respondem na rede |
| `--down` | encerra os *tethers* de todas as placas |
| `--down --stop-server` | e também para o servidor das placas |

## 9.3 O *tether* da licença

O núcleo de FFT é um IP da Intel que a edição gratuita do Quartus só permite usar em modo de avaliação. O bitstream gerado assim (`soc_system_time_limited.sof`) funciona **enquanto o programador JTAG continuar conectado** à placa. Esse processo conectado é o *tether*. Quando ele termina, começa uma contagem de cerca de uma hora, e depois dela **o bitstream inteiro para** — não só a FFT: a convolução passa a esgotar o tempo (medido em 17/09/2026).

O programador do Quartus (`quartus_pgm`), depois de gravar, fica esperando uma tecla no terminal. O `morphe-up.sh` o deixa rodando em segundo plano, numa sessão própria, com uma entrada que nunca fecha, e guarda o identificador do grupo de processos para o `--down`. Há um *tether* por cabo JTAG, em `.morphe-estado/tethers/`, então preparar uma placa não derruba a outra.

Três fatos práticos:

- **o *tether* pertence a uma conta.** Se outra conta tentar reprogramar aquele cabo, o script para e diz de quem ele é;
- **sobrevive ao *logout*** se a estação tiver o *linger* ligado para a conta (`sudo loginctl enable-linger coordenador`, uma vez), mas **não sobrevive a desligar a estação**: depois de cada boot, alguém roda o `morphe-up.sh`;
- para saber se está vivo, de qualquer conta: `pgrep -af quartus_pgm`. Se não imprimir nada, qualquer resultado errado a partir daí é consequência disso.

## 9.4 A instalação de turma

A estação tem duas contas: `coordenador`, com `sudo`, e `alunopds`, sem. A pasta pessoal do coordenador não é legível pelo aluno, então a plataforma mora fora dela, em `/opt/morphe`, como um clone do repositório pertencente ao coordenador e ao grupo dos alunos. O Quartus mora em `/opt/intelFPGA_lite/20.1`, legível por todos.

**Instalar, uma vez.** Na conta do coordenador, copia-se o clone para `/opt/morphe`, sem o ambiente virtual do Python e sem as pastas de compilação do Quartus, e ajustam-se dono e permissões (os comandos estão em `PREPARACAO.md`). O ambiente virtual sai de propósito: o `python3` do sistema já tem NumPy, Matplotlib e SciPy, e o aluno não pode instalar nada.

**Atualizar.** Como `/opt/morphe` é um clone, atualizar é um `git pull`, e o `atualizar.sh` o faz num comando (seção 9.8). Ele mostra o ramo: em 17/09/2026 um `pull` dizia "*Already up to date*" num ramo errado, e duas preparações rodaram com código antigo. Depois de uma atualização que mude o servidor ou o bitstream, rode o `morphe-up.sh --todas` de novo.

**Qualquer conta prepara a placa.** Com o Quartus em `/opt` e a regra do USB-Blaster liberando o cabo para todos, o aluno também roda o `morphe-up.sh`. O preço: o `morphe-up.sh` de uma pessoa reprograma a FPGA e interrompe quem estiver usando aquela placa. Desde 28/09/2026 ele avisa e pergunta antes, quando a placa está em uso; ainda assim, a regra de bolso é rodá-lo para **começar a aula ou consertar uma placa parada**, não durante.

**As duas contas no mesmo estado.** O `.morphe-estado` é compartilhado; o script deixa as pastas com permissão 777 e os arquivos com 666 (o `/opt/morphe` inteiro é fechado para quem não é da turma), para que o que uma conta grava a outra possa reescrever.

**Rodar sempre de `/opt/morphe`.** O cliente lê a placa em `<raiz>/.morphe-estado`, e quem escreve ali é o `morphe-up.sh`. Se os dois rodarem de pastas diferentes — o script de um clone na pasta pessoal, o aplicativo de `/opt/morphe` —, o aluno abre o aplicativo e não acha placa nenhuma. O sintoma parece de rede, e não é.

## 9.5 As chaves SSH

Sem chave, um `morphe-up.sh` pede a senha da placa três vezes. Uma vez por placa e por conta:

```
./morphe-up.sh --setup-ssh --board <ip da placa>
```

A chave criada é **RSA**, e as conexões habilitam o algoritmo `ssh-rsa`. As duas coisas são necessárias juntas: as placas rodam o OpenSSH 6.0, anterior às chaves ed25519 (uma chave ed25519 é gravada na placa e ignorada sem erro), e o OpenSSH 8.9 da estação desabilita por padrão o `ssh-rsa`, que é o único tipo que o servidor antigo assina.

## 9.6 O servidor no boot da placa

Uma vez por placa, instala-se o início automático do servidor (`C/autostart/`, capítulo 4). Com ele, a placa aparece na rede assim que liga — recusando operações até ser preparada — e o cliente não precisa de ninguém que saiba o endereço dela.

## 9.7 O Quartus em todos os computadores

Para que o aluno possa compilar um projeto próprio e programar a FPGA pelo cabo ligado ao computador em que está, cada computador precisa do Quartus **20.1**, da regra do USB-Blaster e do `PATH`. O `instala-quartus.sh`, rodado com `sudo` numa conta que o tenha, faz tudo isso, na ordem:

1. procura Quartus já instalados; uma 20.1 com Cyclone V fora das pastas pessoais é usada como está, uma dentro de uma pasta pessoal é copiada para `/opt`, e só sem nenhuma das duas a cópia vem da estação (ou de um disco externo, com `--origem`);
2. confere a versão e o suporte ao Cyclone V;
3. libera a leitura para todas as contas;
4. instala a regra *udev* do USB-Blaster, para qualquer conta enxergar o cabo;
5. cria a `libudev.so.0`, que o `jtagd` procura;
6. põe a 20.1 na frente do `PATH`, para todas as contas, em `/etc/profile.d/quartus.sh`;
7. cria o atalho "Quartus Prime Lite 20.1" no menu;
8. testa, da conta do aluno, a versão e o cabo.

A versão é sempre a mesma em todo o laboratório porque o Quartus é copiado da estação, e não baixado. O script pode ser rodado de novo sem estragar nada, e `--verificar` só confere. Em 23/09/2026 ele foi rodado em todos os computadores do laboratório.

**Cuidado:** gravar um projeto próprio numa placa que está servindo o Morphe substitui o bitstream da plataforma para todos os que a usam. Depois, `./morphe-up.sh` na estação a restaura.

## 9.8 O aplicativo nos outros computadores

O aplicativo não depende de nada da estação: basta uma cópia do repositório e o `python3` do sistema com NumPy e Matplotlib. O `instala-morphe.sh`, rodado com `sudo` numa conta que o tenha, faz tudo, na ordem, pulando o que já estiver feito:

1. clona o repositório em `/opt/morphe`, na branch principal, ou o atualiza com `git pull` — sem sobrescrever arquivo modificado à mão;
2. deixa como dono quem chamou o `sudo`, para atualizar depois sem `sudo`, e a leitura para todas as contas; abre o `.morphe-estado` para escrita, onde o aplicativo guarda as placas que a varredura achar;
3. instala, se faltarem, os pacotes do Python do sistema (tkinter, NumPy, Matplotlib, SciPy) e confere que nenhum pacote do `pip --user` do aluno esconde os do sistema;
4. cria o atalho "Morphe" no menu de aplicativos;
5. testa da conta do aluno: importa o aplicativo e pergunta às placas, pela porta de estado, o que estão fazendo.

Não há lista de placas para escrever: o aplicativo lê o `placas.conf`, que vem com o repositório. Também não é preciso rodar o `morphe-up.sh` nesses computadores: as placas são preparadas pela estação, e os outros computadores só usam. Com `--verificar`, o instalador só confere, sem mudar nada; rodado de novo, atualiza. Num computador sem clone, o jeito mais simples é clonar numa pasta temporária e rodar o instalador de dentro dela: ele clona para o `/opt/morphe` a partir dessa cópia, sem rede, e aponta o `/opt/morphe` para o GitHub. Em 28/09/2026 o aplicativo foi instalado assim em todos os computadores do laboratório.

**Atualizar os computadores. O aplicativo não se atualiza sozinho.** Ele roda na conta do aluno, que não escreve no `/opt/morphe`, e é bom que não escreva. Depois de uma mudança no repositório, cada computador, e também a estação, precisa de um comando, em qualquer conta com `sudo`:

```
/opt/morphe/atualizar.sh
```

O script descobre qual conta é a dona do `/opt/morphe` (a que rodou o `sudo` na instalação), faz o `git pull` como ela (pedindo a senha do `sudo` se quem roda for outra conta) e mostra a versão de antes, a de agora e a lista do que mudou. Ele só avança a versão: um arquivo modificado à mão nunca é sobrescrito, e nesse caso o script para e diz como ver qual é. Computadores instalados antes de 01/10/2026 ainda não têm o script e precisam, uma única vez, do comando antigo:

```
sudo -u $(stat -c %U /opt/morphe) git -C /opt/morphe pull --ff-only
```

Um aplicativo já aberto continua com a versão antiga até ser fechado. Mudanças no servidor ou no bitstream, além disso, exigem o `morphe-up.sh` na estação: atualizar os computadores só atualiza o aplicativo. Um temporizador do sistema que fizesse a atualização sozinho, ao ligar o computador, está proposto no capítulo 11.

**A armadilha do `pip --user`.** Duas vezes o aplicativo quebrou numa conta por um NumPy ou Matplotlib instalado com `pip install --user`, incompatível com o outro pacote do sistema. Se o aplicativo não abrir com um erro de importação, remova **os dois juntos** — `python3 -m pip uninstall -y matplotlib numpy` — e confira que voltaram a vir de `/usr/lib/python3/dist-packages`. Remover só um deles deixa o outro incompatível e mantém o erro.

## 9.9 A placa na rede certa

As placas pegam endereço por DHCP, da rede do laboratório: hoje, a faixa `172.16.230.x`. Se o cabo de rede de uma placa for ligado num ponto de outra rede — outro switch, uma tomada de outra sala, um roteador pequeno na bancada —, ela recebe o endereço *daquela* rede e some para a estação e para os alunos. Foi o que aconteceu em 28/09/2026: a placa 2 subiu com `192.168.0.108`.

O `morphe-up.sh` mostra esse caso sem ambiguidade: a FPGA é programada ("*Configuration succeeded*", porque o JTAG não depende da rede) e em seguida ele para com "não há caminho de rede até" a placa. O *tether* fica vivo, então não há pressa. Para resolver:

1. **Ver o endereço que a placa pegou**, pelo console serial (`screen /dev/ttyUSB0 115200`, conta `root`): `ip addr show eth0`. Se não for `172.16.x.x`, é a rede errada. Quem entregou o endereço aparece em `grep -h dhcp-server-identifier /var/lib/dhcp/dhclient*.leases`.

2. **Trocar o cabo de lugar**: ligá-lo no mesmo switch ou ponto de rede da outra placa, que está funcionando.

3. **Pedir um endereço novo, sem reiniciar a placa**, no console serial: `ifdown eth0` e depois `ifup eth0`. Conferir de novo com `ip addr show eth0`.

4. **Terminar a preparação sem reprogramar a FPGA**, na estação: `./morphe-up.sh --skip-fpga --board <ip>`. O `--skip-fpga` preserva o *tether*, que já está contando desde a programação.

Se o endereço novo for diferente do anterior, mas dentro da rede certa (o DHCP pode trocá-lo), não há nada a fazer: o aplicativo e o `morphe-up.sh` acham a placa por *broadcast*, pelo nome, no endereço que ela tiver (capítulo 8). A reserva de DHCP na TI, antes proposta, deixou de ser necessária. O *broadcast* só alcança a rede do laboratório: uma placa em outra rede continua invisível, e é o caso desta seção.

## 9.10 Uma placa nova

Todas as DE1-SoC saem de fábrica com **o mesmo endereço MAC** (`12:34:56:78:90:12`) e os mesmos endereços IP fixos auxiliares. Duas placas na mesma rede disputam o endereço, e o sintoma — conexões intermitentes — parece defeito de placa. Por isso incorporar uma placa não é só ligá-la. Os passos, detalhados em `docs/NOVA-PLACA.md`:

1. **clonar o cartão SD** de uma placa que funciona, cuidando do tamanho: a imagem tem o tamanho do cartão de origem, e um cartão nominalmente igual pode ser menor;

2. **trocar o MAC no U-Boot**, pelo console serial (`setenv ethaddr 02:00:00:6d:70:NN` e `saveenv`, com $NN$ o número da placa), e dar à placa um nome próprio;

3. **limpar o que o clone trouxe**: as concessões de DHCP da placa de origem e os endereços fixos de fábrica; configurar a rede como `auto eth0`;

4. instalar a chave SSH de cada conta e o início automático do servidor;

5. preparar com o `morphe-up.sh`, com o cabo e o endereço dela.

## 9.11 Sintoma, causa e o que fazer

Tabela 9.2: Sintomas da operação do laboratório.

| **Sintoma** | **Causa provável** | **O que fazer** |
|---|---|---|
| Operações dão tempo esgotado, ou a FFT volta errada, depois de funcionar | o *tether* caiu há mais de uma hora | `pgrep -af quartus_pgm`; `./morphe-up.sh` |
| "FPGA não preparada" | a placa foi ligada depois da última preparação | `./morphe-up.sh` |
| "o bitstream na FPGA não é o deste servidor" (ADC) | servidor novo com bitstream antigo | `./morphe-up.sh` com o bitstream que tem o ADC |
| O aplicativo abre e não acha placa | as placas não foram preparadas, ou o aplicativo está desatualizado | `estado_placas.py`; `./morphe-up.sh --todas`; `atualizar.sh` |
| `Permission denied` no `.morphe-estado` | arquivo antigo gravado por outra conta | corrigir dono e permissões uma vez (`PREPARACAO.md`); nunca `sudo` |
| A senha da placa é pedida várias vezes | falta a chave desta conta | `--setup-ssh` |
| O `atualizar.sh` para com "a atualização falhou" | sem rede até o GitHub, ou arquivo modificado à mão no `/opt/morphe` | conferir a rede; `git -C /opt/morphe status` |
| `qsys-generate: not found` ou Quartus fora do `PATH` | terminal em `ksh`, que não lê o `.bashrc` | digitar `bash` antes |
| O aplicativo não abre, com erro de importação | pacote instalado com `pip --user` | remover NumPy e Matplotlib juntos |
| "não há caminho de rede até" a placa, logo depois de programar a FPGA | o cabo de rede da placa está em outra rede (endereço fora de `172.16.x.x`) | seção 9.9 |
| A placa some da rede depois de ligada | cartão com servidor anterior a 21/09/2026 no início automático | atualizar o servidor da placa |

# Capítulo 10

# Desenvolvimento e verificação

Este capítulo é para quem vai mudar o sistema: acrescentar uma operação, alterar um acelerador ou recompilar o hardware. Ele descreve o processo que foi usado para o IIR e para o ADC, os testes que existem e as regras que evitam os erros que já aconteceram.

## 10.1 Três regras antes de começar

**O bitstream, o `hps_0.h` e o servidor são um conjunto.** Misturar um bitstream de uma versão com um `hps_0.h` de outra **não produz erro**: o servidor escreve em endereços válidos, porém errados, e devolve zeros ou lixo. Foi essa a causa de dias de investigação equivocada durante a expansão para 1024 pontos. O comando `python3 Quartus/gen_hps_header.py --check` confere, sem Quartus, se o `hps_0.h` corresponde ao `soc_system.sopcinfo` ao lado; é a primeira coisa a rodar diante de um resultado inexplicável.

**O `hps_0.h` é gerado, nunca editado.** Um cabeçalho escrito à mão descreve um hardware que não existe. Ele sai do `.sopcinfo`, que sai do Platform Designer. Até para um teste rápido, gere.

**O repositório manda, não a placa.** Quando o que está numa placa diverge do repositório, o certo é descobrir por quê e levar a correção ao repositório — nunca copiar da placa para o repositório o que não se sabe de onde veio. Em 15/09/2026, na primeira compilação feita inteiramente a partir do repositório, a IFFT travou: o `fft_wrapper.v` versionado não era o que tinha gerado o bitstream em uso, porque uma correção do autor existia só numa cópia na estação. Até ali o repositório reproduzia o bitstream, mas não a compilação. O `PROVENIENCIA.sha256` (seção 10.6) existe para que isso não se repita.

## 10.2 Recompilar o bitstream

O projeto do Quartus está em `Quartus/`: o sistema do Platform Designer (`soc_system.qsys`), o topo (`ghrd_top.v`), os aceleradores e as restrições (`soc_system.qsf`, `soc_system_timing.sdc`). A compilação é feita na estação, no clone de desenvolvimento da conta do coordenador — onde ficam as pastas de trabalho do Quartus — e não em `/opt/morphe`. Os guias passo a passo são `docs/COMPILAR-IIR.md` e `docs/COMPILAR-ADC.md`; o roteiro geral é este:

1. **Simular primeiro.** Se o acelerador tem testbench, ele tem de passar antes de gastar meia hora de compilação (seção 10.3).
2. **Gerar o sistema.** Depois de mudar o `.qsys`:

```
qsys-generate soc_system.qsys --synthesis=VERILOG --family="Cyclone
V--part=5CSEMA5F31C6
```

3. **Regenerar os endereços:** `python3 gen_hps_header.py`. Conferir que os símbolos novos apareceram no `C/hps_0.h`; se algum faltar, o Platform Designer não gerou o módulo.

4. **Compilar** (cerca de 30 minutos): `quartus_sh --flow compile soc_system.qpf`.

5. **Conferir recursos e tempo.** Em `output_files/` ficam o resumo de ocupação, `soc_system.fit.summary`, e a análise de tempo, `soc_system.sta.rpt`, em que a frequência máxima que vale é a do modelo "*Slow 1100mV 85C*". Ela precisa ficar acima de 50 MHz. O `relatorio_timing.tcl` lista os caminhos críticos quando não fica.

6. **Programar e testar o que já funcionava**, antes do que é novo: `./morphe-up.sh` (que recompila o servidor, porque o `hps_0.h` mudou) e os testes da seção 10.4.

7. **Registrar.** Regerar a proveniência, versionar o bitstream, o `.sopcinfo` e o `hps_0.h` gerados, e anotar os números no `docs/DIARIO.md`.

**Armadilhas conhecidas.**

- Os `iir_*.v` **não estão listados** no `.qsf`: o Quartus os acha por estarem na pasta do projeto. Movê-los quebra a compilação.
- Depois de um `qsys-generate`, o bitstream *tem* de ser recompilado e reprogramado: o `sysid` muda, e o servidor passa a recusar o ADC com o bitstream antigo (seção 4.3). É o comportamento desejado.
- No clone de desenvolvimento da estação, nunca use `git reset --hard`: há arquivos de trabalho do Quartus rastreados por engano.
- Alguns terminais da estação abrem em `ksh`, que não lê o `.bashrc`: se um comando do Quartus der `not found`, digite `bash` antes.

**Quando o tempo não fecha.** Foi o que aconteceu na primeira compilação do IIR (49,11 MHz, 14/09/2026). O `relatorio_timing.tcl` mostrou que os quinze piores caminhos eram o mesmo: num único ciclo, o multiplexador que escolhe os operandos da seção corrente entre as 16, as cinco multiplicações de 32 bits, a soma de 72 bits, o arredondamento, a saturação e a escrita de volta — 19,7 ns de lógica em 20 ns. A correção foi um estado `S_SEL` no `iir_cascade.v`, que copia os operandos para registradores antes do cálculo; o registrador no meio da lógica é o que encurta o caminho (um simples estado de espera não mudaria nada, porque o analisador mede de registrador a registrador). A latência subiu de 24 para 35 ciclos por amostra, o resultado não mudou — o mesmo teste bit a bit provou isso — e a compilação seguinte fechou a 60,99 MHz.

## 10.3 Testbenches

Os aceleradores escritos no estágio têm testbench em Icarus Verilog, que roda no Linux e no Windows sem Quartus.

`Quartus/testbench/roda_tb_iir.sh` simula, contra vetores do `gera_vetores_iir.py` (em `Python/ferramentas`), tirados do modelo em ponto fixo do `iir_design.py`, uma seção, a cascata e o estado entre execuções: três blocos encadeados pelo estado têm de dar a saída do sinal inteiro de uma vez. Os testbenches são `tb_iir_sos.v`, `tb_iir_cascade.v` e `tb_iir_estado.v`. O critério é igualdade **bit a bit** em todas as amostras.

`Quartus/testbench/roda_tb_adc.sh` simula o `adc_captura.v` contra um modelo do LTC2308 que confere os tempos da folha de dados: captura única a várias taxas, divisor abaixo do mínimo, memória cheia, captura contínua com a memória dando a volta, parada no meio de um quadro, e captura única depois da contínua. Confere o período exato, o canal, a ordem dos bits e, na contínua, que o contador corresponde à última amostra gravada.

`Quartus/testbench/roda_tb_conv.sh` simula o `conv1d.v` (convolução e FIR) com 16 e com 1024 amostras aleatórias e confere as saídas, uma a uma, contra a conta inteira exata arredondada; mostra também o erro que o truncamento antigo daria. Os `memory_*_controller.v` escrevem literais como `1'b_0`, que o Quartus aceita e o Icarus não: o script simula cópias corrigidas, sem tocar nos originais.

`Quartus/testbench/tb_audio_i2s.v`, `tb_audio_vu.v` simulam a interface I2S do codec contra um modelo do WM8731 (recepção, *loopback* e tom) e o medidor de nível.

Os dois terminam imprimindo `TUDO OK` ou a primeira divergência.

## 10.4 Testes contra a placa

As ferramentas de teste ficam em `Python/ferramentas/` e rodam de dentro de `Python/`, com o endereço da placa como argumento. Todas conferem resultados contra uma referência, e não só a ausência de erro.

Tabela 10.1: Os testes contra a placa.

| **Ferramenta** | **O que confere** |
|---|---|
| `Quartus/morphe_ping.py` | os quatro testes do `morphe-up.sh`: conexão, protocolo, convolução de impulso, FFT de impulso |
| `testa_bit_inverse.py` | o bit `inverse` do núcleo da FFT e a convenção $1/N$ |
| `testa_ifft_roundtrip.py` | sinal → FFT → IFFT → sinal, contra o NumPy |
| `testa_iir_hw.py` | o IIR da placa contra o modelo em ponto fixo, bit a bit, em vários filtros |
| `testa_blocos.py` | o processamento em blocos (sem placa, contra o NumPy; com `--placa`, contra a placa) e a leitura de arquivos |
| `testa_concorrencia.py` | vários clientes ao mesmo tempo, conferindo cada resposta (capítulo 8) |
| `testa_adc.py` | o ADC em degraus: `basico` (sem nada ligado), `tensao` (com multímetro), `senoide` (com gerador), `continuo` (captura longa, conferindo as emendas) e `canais` |
| `compara_professor_fpga.py` | o `dsp_iir_filter.m` do Prof. Sanca contra a placa, separando o custo do ponto fixo do que é limitação do MATLAB |

Depois de qualquer bitstream novo, a sequência mínima é o `morphe-up.sh` (quatro testes), `testa_ifft_roundtrip.py`, `testa_iir_hw.py` e `testa_blocos.py --placa`. Foi o que se rodou em 24/09/2026 com o bitstream do ADC, antes de testar o ADC: nada do que já existia mudou.

**O tempo como diagnóstico.** Cada operação tem um tempo típico na placa (cerca de 170 ms na FPGA para uma convolução de 1024 por 1024, cerca de 21 ms de ponta a ponta para a FFT), e o servidor o registra em cada operação. Um tempo muito menor é sinal de defeito, não de sorte: um bitstream mal compilado já foi visto terminar a convolução em 3,3 ms sem ter calculado nada.

**Diagnóstico sem o servidor.** Quando é preciso saber se o problema está no servidor ou no hardware, dá para escrever e ler as memórias e os registradores direto pelo `/dev/mem`, na placa. O procedimento está no item 15 do `docs/RESSALVAS.md`.

## 10.5 Sem placa: o servidor em simulação

O servidor em C pode ser exercitado sem placa. Em 24/09/2026, para o ADC, o `morphe_server.c` foi compilado num Linux x86 com o `/dev/mem` trocado por memória comum e uma *thread* fazendo o papel do `adc_captura.v`. Isso permitiu testar a captura contínua (sequências idênticas amostra por amostra em 100 mil amostras, perda forçada, cliente que fecha a conexão no meio) antes da primeira compilação do bitstream. O mesmo arranjo serve para qualquer operação nova: o servidor não sabe que a memória não é a da FPGA.

## 10.6 A proveniência

O `PROVENIENCIA.sha256`, na raiz, lista o *hash* SHA-256 do conjunto inseparável: os fontes de hardware que viram bitstream (`conv1d.v`, `fft_wrapper.v`, `iir_*.v`, `adc_captura.v`, `ghrd_top.v`, `soc_system.qsys`), o `.sopcinfo`, o bitstream, o `hps_0.h`, o `morphe_config.h` e o servidor. O cabeçalho diz em que máquina, de que *commit* e com que validação ele foi gerado.

Conferir um clone é um comando, da raiz:

```
sha256sum -c PROVENIENCIA.sha256
```

Regerar é tarefa de quem compilou e validou, na máquina onde o fez, logo depois da validação:

```
./gera_proveniencia.sh "morphe_ping 4/4; IFFT 0 falhas; Fmax 61,56 MHz"
```

O script se recusa a rodar sem a frase de validação: proveniência sem validação não é proveniência. Ele existe porque o arquivo, editado à mão, ficou desatualizado duas vezes.

## 10.7 Fluxo de trabalho com o `git`

- O repositório de trabalho é o *fork* `nicassiosantos/morphe` no GitHub. O ramo principal do estágio é `estagio/v1.1-1024pontos`; cada mudança de hardware nasce num ramo próprio (`estagio/v1.3-iir-hw`, `estagio/v1.7-adc`) e só é mesclada depois de validada na placa. Enquanto isso, o bitstream do ramo principal continua valendo.
- Na estação, o clone de desenvolvimento e o `/opt/morphe` só recebem mudanças por `git pull`. O que for gerado na estação (bitstream, `.sopcinfo`, `hps_0.h`) é versionado de lá.
- Antes de um `pull` em `/opt/morphe`, confira o ramo (`git branch -vv`).
- Toda validação vai para o `docs/DIARIO.md`, com o número medido e onde foi medido; trabalho escrito e não validado fica marcado como tal.

## 10.8 Acrescentar um acelerador

Juntando tudo, o caminho de um acelerador novo, na ordem que evita retrabalho:

1. um modelo em Python, em ponto fixo, que defina o resultado esperado bit a bit — e o estudo de viabilidade numérica nele, antes de qualquer Verilog (foi assim com o IIR: o `estuda_iir_ponto_fixo.py` respondeu se o Q15.16 bastava antes de o RTL existir);
2. o RTL, obedecendo ao protocolo comum de `start` e `done` (seção 3.3), e o seu testbench contra o modelo;
3. memórias e registradores no Platform Designer, nos primeiros endereços livres, sem mover os existentes; a instância no `ghrd_top.v`;

4. a compilação (seção 10.2);
5. a operação no protocolo e no servidor (seção 5.7);
6. um teste contra a placa, em `Python/ferramentas/`;
7. a janela, por último.

# Capítulo 11

# Limitações e trabalho futuro

Este capítulo reúne o que o sistema não faz hoje, cada item com o motivo e, quando há, o caminho para resolvê-lo. Nada aqui impede o uso em aula; são os pontos de partida para quem continuar o trabalho.

## 11.1 Hardware

**Tamanho fixo de 1024 amostras.** Os aceleradores foram sintetizados para blocos de 1024. Sinais maiores são tratados no cliente, por blocos (seção 7.7), com o custo de uma requisição por bloco. Aumentar o bloco exige memórias maiores, um núcleo de FFT gerado de novo e, para a convolução, aceitar um tempo que cresce com o quadrado do tamanho: 1024 por 1024 já leva 170 ms.

**A licença de avaliação da FFT.** O bitstream só funciona com o *tether* vivo, e para uma hora depois de ele cair (seção 9.3). O `morphe-up.sh` torna isso invisível na rotina, mas a estação precisa ficar ligada durante a aula e o `morphe-up.sh` tem de rodar depois de cada boot dela. As saídas são uma licença do núcleo da Intel, ou uma FFT própria em Verilog, que tiraria a restrição e daria controle sobre a aritmética interna (hoje o teto de 92 dB é do núcleo).

**A aritmética do `conv1d`.** Desde 15/10/2026 a convolução e o FIR arredondam, com erro de meio LSB (seção 3.3.2). O estouro continua silencioso: a soma dá a volta em vez de saturar. Saturar, como no IIR, custaria pouco e tiraria a última surpresa do bloco. O bitstream oficial com o arredondamento, compilado no Quartus 20.1 da estação, ainda não foi gerado; o que foi medido usou um bitstream de teste do Quartus 23.1.

**A aritmética dentro do núcleo da FFT.** O núcleo da Intel é criptografado: dá para ver que ele tem uma unidade de arredondamento e saturação, mas não em que modo ela trabalha, nem mudá-la.

**Restos do projeto original.** O IP de FIR da Intel (`fir_ii`) e o seu adaptador continuam no projeto sem instância, e o registrador `fir_error` não é acionado por nada. Podem sair numa próxima compilação, junto com a conferência de que nada mudou.

## 11.2 Aquisição

**O que falta validar.** Na data deste manual o ADC foi validado na placa com tensões contínuas (0 V e 3,3 V, contra o valor nominal), com a captura contínua a 200 kHz sem perda e, em 07/10/2026, com uma senoide de 1 kHz do gerador de funções (seção 3.4.4). Falta a comparação

da leitura em 3,3 V com um multímetro. Com a senoide, falta registrar na placa o modo *ao vivo* da janela de aquisição e o analisador espectral, que em 30/09/2026 foi testado na placa só com a entrada sem sinal.

**ENOB com pouca folga.** Com o gerador, a captura contínua mediu ENOB de 9,03 bits (SINAD de 56,1 dB): passa no critério do `testa_adc.py` (acima de 9), mas fica longe dos cerca de 11,9 bits que o LTC2308 promete. A medida inclui o ruído e as harmônicas do próprio gerador, e os dois não foram separados. Para saber quanto é do conversor, seria preciso um gerador de menor distorção ou um filtro passa-baixas na entrada.

**Um transitório no início da captura.** Com a entrada do canal 0 da placa 1 solta, toda captura começa perto de 2,4 V e decai, em 40 a 50 amostras, até um nível estável que depende da taxa (1,96 V a 10 kHz, 1,84 V a 50 kHz). Parece efeito da entrada sem nada ligado, de alta impedância, que o próprio conversor carrega. Se o transitório continuar com o gerador ligado, é defeito da captura, afeta também a janela de aquisição, e precisa ser investigado.

**Não há filtro *anti-aliasing*.** O sinal chega ao conversor sem filtro analógico. Um tom acima de $f_s/2$ aparece dobrado no espectro. Para medir, isso é uma limitação; para ensinar amostragem, é uma oportunidade.

**Faixa de entrada.** A entrada lê de 0 a 4,095 V e pode ser danificada abaixo de −0,3 V. Um sinal alternado precisa de deslocamento DC antes do conector. Um pequeno circuito de condicionamento (deslocamento, proteção e filtro) tornaria a entrada segura para uso direto pelos alunos.

**Um canal por captura.** A captura lê um canal, ou um par diferencial, por vez. Como o LTC2308 recebe a palavra de configuração a cada conversão, alternar dois canais é uma mudança pequena no `adc_captura.v` (e exige recompilar); abriria correlação cruzada e estimativa de atraso.

**A placa fica ocupada durante a captura.** A captura contínua mantém a conexão aberta, e o servidor não atende mais ninguém nesse tempo (capítulo 8). Limitar a duração, ou atender outras operações durante a captura, são decisões ainda em aberto.

## 11.3 Codec de áudio

**Áudio e ADC dividem a memória.** Como o áudio usa a memória e os registradores do ADC (seção 3.5.3), os dois não capturam ao mesmo tempo, e o servidor atende uma operação por vez: enquanto a janela Áudio captura ou faz *loopback* ao vivo, a placa fica ocupada. Uma memória própria para o áudio, numa próxima regeneração do Platform Designer, tiraria essa restrição.

**O som ainda não passa pelos aceleradores.** O áudio chega ao computador e volta, mas os filtros da placa não o processam em tempo real: o próximo passo natural é um filtro no caminho do áudio, entre a entrada e a saída do codec, com os coeficientes vindos do aplicativo.

**Uma entrada por vez.** O codec lê a LINE-in ou o MIC-in, conforme o `morphe-up.sh` o configurou (`--codec`); trocar exige rodá-lo de novo, ou o `./codec` na placa.

**Taxa fixa.** Sem PLL, a única taxa é 48 828,125 Hz. Outras taxas do codec (8, 32, 44,1, 96 kHz) exigem outro relógio-mestre.

## 11.4 Infraestrutura

Descritas no capítulo 8. Três melhorias resolveram a maior parte do que faltava: a **porta de estado** (28/09/2026) diz a qualquer momento o que cada placa está fazendo, e o cliente escolhe a placa livre e menos usada; o `placas.conf` versionado (28/09/2026) dá a todos os computadores a mesma lista de placas; e a **busca por *broadcast*** (30/09/2026) acha as placas pelo nome, no IP que tiverem, o que tornou desnecessária a reserva de DHCP. O que continua faltando: **reserva** de placa (a escolha divide a turma, mas não impede duas pessoas na mesma placa) e **troca automática de placa** durante o uso. Falta também testar dois computadores abrindo o aplicativo ao mesmo tempo: devem cair em placas diferentes.

A mesma porta de estado protege quem está usando uma placa: o `morphe-up.sh` a consulta antes de reprogramar e pergunta se deve continuar quando a placa está em uso. Mas a pergunta só funciona para quem roda o script; nada impede, por exemplo, alguém de gravar um projeto próprio pelo Quartus numa placa que está servindo a turma.

**Atualização dos computadores.** Em 28/09/2026 o Quartus e o aplicativo estavam em todos os computadores, cada um instalado pelo seu script (`instala-quartus.sh` e `instala-morphe.sh`). Mas o aplicativo não se atualiza sozinho: desde 01/10/2026 basta um comando por computador, `/opt/morphe/atualizar.sh` (seção 9.8), mas ainda é preciso lembrar de rodá-lo em cada um, e com vários computadores isso acaba esquecido. A proposta é um temporizador do sistema (*systemd timer*), instalado pelo próprio `instala-morphe.sh`, que atualize o `/opt/morphe` como a conta dona, ao ligar o computador e a cada hora, só por avanço simples (`--ff-only`): um arquivo modificado à mão continua protegido, porque nesse caso a atualização é recusada.

**A estação.** Três coisas dependem dela: o *tether* da licença (ela precisa ficar ligada durante a aula, e o `morphe-up.sh` tem de rodar depois de cada boot), os cabos JTAG das placas, e a compilação do bitstream. O servidor SSH que ela ganhou em 23/09/2026 só servia para os outros computadores copiarem o Quartus, e foi desligado em 30/09/2026.

## 11.5 Ideias que a plataforma já permite

Com o que existe hoje, sem mudar o hardware:

- aula de amostragem e *aliasing*, capturando um tom do gerador acima e abaixo de $f_s/2$;
- com o analisador espectral (seção 7.4): o espelho em $f_s - f$, o efeito da janela na amplitude de um tom, a resolução $f_s/N$ separando dois tons próximos;
- medir o próprio conversor (SINAD, ENOB) e comparar com os 74 dB teóricos de 12 bits;
- multitaxa: capturar rápido e dizimar com o FIR da placa;
- os exercícios do Oppenheim que usam sequências dadas, agora que o aplicativo lê sinais de arquivo de qualquer tamanho;
- estudar o efeito da quantização dos coeficientes de um IIR de ordem alta em forma direta contra seções de segunda ordem, com o projetista e o comparador.

# Apêndice A

# Estrutura do repositório

A árvore abaixo mostra o que importa em cada pasta. A separação entre o que é produto (o que roda quando alguém usa o Morphe) e o que não é está também no `README.md` da raiz.

```
morphe/
├── README.md ..... ponto de entrada
├── morphe-up.sh ..... prepara a placa: FPGA, servidor, marca
├── instala-quartus.sh ..... Quartus 20.1 para todas as contas de um PC
├── instala-morphe.sh ..... o aplicativo em /opt/morphe, com atalho
├── placas.conf ..... as placas do laboratório
├── deploy.sh, gera_proveniencia.sh, PROVENIENCIA.sha256
├── Python/ ..... o aplicativo
│   ├── morphe_app.py ..... janela principal
│   ├── *_window.py ..... uma janela por operação
│   ├── morphe_protocol.py ..... cliente TCP
│   ├── dsp_core.py, blocos.py ..... ponto fixo, referência, sinais longos
│   ├── fir_design.py, iir_design.py ..... projetistas de filtros
│   ├── aquisicao.py, sinal_arquivo.py ..... ADC e sinais de arquivo
│   └── ferramentas/ ..... testes contra a placa, figuras, vetores
├── C/ ..... o servidor da placa
│   ├── morphe_server.c, morphe_protocol.h, morphe_config.h
│   ├── hps_0.h ..... gerado do Platform Designer
│   └── autostart/
├── Quartus/ ..... o hardware
│   ├── ghrd_top.v ..... topo: instancia os aceleradores
│   ├── conv1d.v, fft_wrapper.v, iir_*.v ..... os aceleradores
│   ├── adc_captura.v ..... o controlador do ADC
│   ├── soc_system.qsys, soc_system.qsf
│   ├── tb_*.v, roda_tb_*.sh ..... testbenches (IIR e ADC)
│   ├── gen_hps_header.py ..... gera o hps_0.h
│   └── output_files/ ..... o bitstream
├── exemplos/ ..... pacotes .mrph e sinais de exemplo
├── docs/ ..... documentação técnica, diário e este manual
└── legado/ ..... do projeto original, fora do produto
```

Três regras sobre esta estrutura:

- **os `iir_*.v` não saem de `Quartus/`**: não estão listados no `soc_system.qsf`, e o Quartus os acha por estarem na pasta do projeto;
- **o `C/hps_0.h` e o `Quartus/soc_system.sopcinfo` são gerados**, e vão juntos com o bitstream que os gerou;

- **as ferramentas rodam de dentro de `Python/`**, porque o `ferramentas/_caminho.py` acrescenta a pasta de cima ao caminho de importação; por exemplo: `python3 ferramentas/testa_iir_hw.py <ip>`.

Tabela A.1: A documentação técnica, em `docs/` e na raiz.

| **Arquivo** | **Assunto** |
|---|---|
| `PREPARACAO.md` | preparar a placa, instalar para a turma, Quartus nos computadores |
| `docs/GERENCIAMENTO-PLACAS.md` | várias placas e vários clientes |
| `docs/PRECISAO-NUMERICA.md` | os formatos de ponto fixo e os erros medidos |
| `docs/ADC.md` | o conversor da placa e como a captura funciona |
| `docs/COMPILAR-IIR.md`, `docs/COMPILAR-ADC.md` | recompilar o bitstream, passo a passo |
| `docs/NOVA-PLACA.md` | pôr uma placa nova em serviço |
| `docs/RESSALVAS.md` | o que foi corrigido no projeto original, e por quê |
| `docs/LINHA-DE-BASE-PASSOS.md` | a medição de usabilidade antes das mudanças |
| `docs/DIARIO.md` | o diário do estágio: o que foi feito, medido e validado, dia a dia |
| `docs/manual/` | este manual |

# Apêndice B

# Referência rápida

Os comandos do dia a dia. Os da estação rodam de `/opt/morphe`, na conta de quem vai preparar a aula; nunca com `sudo`.

## Preparar as placas (estação)

```
cd /opt/morphe && ./morphe-up.sh --todas
```

Só uma placa:

```
./morphe-up.sh --cable 'DE-SoC [1-2]'
./morphe-up.sh --cable 'DE-SoC [1-3]'
```

Ver o que está no ar, e se os *tethers* estão vivos:

```
./morphe-up.sh --status
pgrep -af quartus_pgm
```

## Abrir o aplicativo (qualquer computador)

```
cd /opt/morphe/Python && python3 morphe_app.py
```

## Uma vez por placa e por conta

```
./morphe-up.sh --setup-ssh --board <ip>
```

## Atualizar (estação e qualquer computador)

Numa conta com `sudo`, ou na conta dona do `/opt/morphe`:

```
/opt/morphe/atualizar.sh
```

Na estação, se o servidor ou o bitstream mudaram, `./morphe-up.sh --todas` de novo. Computador instalado antes de 01/10/2026, uma única vez:

```
sudo -u $(stat -c %U /opt/morphe) git -C /opt/morphe pull --ff-only
```

## Os outros computadores

Instalar, numa conta com `sudo` (uma vez por computador):

```
git clone -b estagio/v1.1-1024pontos https://github.com/nicassiosantos/morphe.git
/tmp/morphe-inst
```

```
sudo /tmp/morphe-inst/instala-morphe.sh
```

Atualizar: `/opt/morphe/atualizar.sh`, como acima.
Só conferir, sem mudar nada: `sudo /opt/morphe/instala-morphe.sh --verificar`.

## O que cada placa está fazendo

```
python3 /opt/morphe/Python/ferramentas/estado_placas.py --seguir
```

## Conferir um clone

```
sha256sum -c PROVENIENCIA.sha256
```

```
python3 Quartus/gen_hps_header.py --check
```

## Testar contra a placa (de dentro de `Python/`)

```
python3 ../Quartus/morphe_ping.py <ip>
```

```
python3 ferramentas/testa_ifft_roundtrip.py <ip>
```

```
python3 ferramentas/testa_iir_hw.py <ip>
```

```
python3 ferramentas/testa_blocos.py --placa <ip>
```

```
python3 ferramentas/testa_adc.py <ip> basico
```

```
python3 ferramentas/testa_concorrencia.py --modo mesma --clientes 6
```

## Encerrar

```
./morphe-up.sh --down
```

encerra os *tethers*; em uma hora as placas param. Com `--stop-server`, para também os servidores.

## Quando algo dá errado

| **Sintoma** | **Primeiro passo** |
|---|---|
| tempo esgotado, FFT errada | `pgrep -af quartus_pgm`; `./morphe-up.sh` |
| “FPGA não preparada” | `./morphe-up.sh` |
| aplicativo não acha placa | `estado_placas.py`; `./morphe-up.sh --todas` |
| resultado inexplicável | `gen_hps_header.py --check` |
| aplicativo não abre | remover NumPy e Matplotlib do `pip --user` |

As tabelas completas estão nos capítulos 8 e 9.